RÉPUBLIQUE DU CAMEROUN
****
Paix - Travail - Patrie
****
UNIVERSITÉ DE YAOUNDÉ I
****
FACULTÉ DES SCIENCES
****
DÉPARTEMENT D'INFORMATIQUE
****

REPUBLIC OF CAMEROON
****
Peace - Work - Fatherland
****
UNIVERSITY OF YAOUNDE I
****
FACULTY OF SCIENCE
****
DEPARTMENT OF COMPUTER SCIENCE
****

KIMVIEWARE :A GENERIC MICROSERVICE PLATFORM FOR SOFTWARE TESTING FRAMEWORK

With a view to obtaining a Master's degree in Computer Science Research

Option :

Software Engineering and Information Systems

Presented by :

NDEMAFO NKENANG FLAVIE DAVILA

Registration number :

19M2267

Supervised by :

Dr. DJAM Xaveria Youh - KIMBI

Academic year 2024/2025

University of Yaounde I
Department of Computer Science

KIMVIEWARE : A GENERIC MICROSERVICE PLATFORM FOR SOFTWARE TESTING FRAMEWORK

Presented by :
NDEMAFO MKENANG FLAVIE DAVILA

Registration number :
19M2267

Supervised by :
Dr. DJAM KIMBI Xaveria Youh

Academic year 2024/ 2025

Certification

This is to certify that **NDEMAFO NKENANG FLAVIE DAVILA 19M2267** of the University of Yaoundé I, Department of Computer Science, Specialty Information System and Software Engineering, is the author of this research work carried out under the supervision of Dr. Djam Xaviera Youh KIMBI in partial fulfillment of the award of a Master's Degree in Computer Science.

Sign : . . . . . .

Date : . . . . . .

DEDICATIONS

This research work is dedicated to :

My parents, **Mr. NDEMAFO DAVID** and **Mrs. MELAGO FLORE**,

My tutors, **Monsieur and Madame LEKEUFACK**,

My uncle, **PAPA MELAGO MEKONTCHOUE ROMEYO BLASIS**,

ACKNOWLEDGEMENT

I wish to express my deepest gratitude to all those who contributed, directly or indirectly, to the successful completion of this thesis. Their guidance, support, and patience were invaluable in navigating the complexities of this research on test suite optimization.

First and foremost, my sincere appreciation goes to my supervisor, **Dr. DJAM Xaveria Youh KIMBI**. Thank you for accepting to oversee this work despite your demanding schedule. Your rigorous scientific approach, wealth of expertise, and unwavering confidence were instrumental in shaping the methodology and ensuring the academic quality of the **KIMVIEware** project. Furthermore, your inspiring guidance helped broaden my perspective on the research problem and deepen my theoretical understanding of meta-heuristic optimization. Your constant encouragement and patience throughout the conceptualization and writing process made this endeavor possible.

I extend my gratitude to the distinguished members of the jury for dedicating their time and expertise to review and evaluate this research. Your insightful questions and feedback will undoubtedly enhance the final contribution of this work to the field of software testing.

My thanks also go to the faculty and staff of the **Computer Science Department at the University of Yaoundé I**. Their commitment to excellence, profound knowledge, and mentorship over the years have laid the essential academic foundation for this work. Their inspiration continues to guide my professional development.

I am particularly thankful to the **Worldvoice Group** company for providing an exceptional environment for the practical implementation of this research. The practical supervision, technical resources, and professional setting offered by the group were crucial in developing and validating the microservice architecture and the **EvoPath-GA** algorithm.

Finally, a heartfelt thanks to my friends and classmates—**DJEUMENI DJOMBISSIE LEVINNE CLEMENCE**, **FOUATSOP PATRICK**, **EBA CHANTAL**, **KEMTHO ZIDANE**—for their moral support, invaluable insights, and collaborative spirit. Their presence and assistance during the challenging phases of this work provided essential relief and motivation. I also thank my minor sisters and my family and loved ones for their enduring belief and unconditional support throughout my academic career.

ABSTRACT

Context and Problem. Software testing accounts for over 50% of modern development budgets. This thesis addresses three persistent issues limiting automated testing effectiveness: 1) **Path Explosion** in symbolic execution (formally demonstrated, e.g., $262,144$ paths for six rooms in a scheduling simulation), 2) **Test Suite Redundancy** from automatic generation, and 3) **The Research-Practice Gap** due to academic prototypes' lack of scalability and CI/CD integration.

Main Contributions. This thesis introduces **KIMVIEware**, a generic platform built on a microservice architecture. KIMVIEware tackles these challenges through a novel four-phase pipeline integrating symbolic execution, heuristic reduction, multi-objective genetic optimization, and operational feedback.

SGATS Reduction Algorithm (Phase 2). SGATS (**S**election-**G**uided **A**ggregation of **T**rajectories **S**ymbolic) is a polynomial-time algorithm ($O(N^2 \cdot L)$) that eliminates structural and semantic redundancy. It uses a priority function $\rho(t)$ and a similarity function $sim(t_i, t_j)$ (LCP and Hamming distance). The **Fusion Hit** detection leverages symbolic state equivalence for dynamic subsumption. Experimental results show an average reduction rate of **85.1**% with **100**% **coverage preservation** (Theorem 3.3: $\mathbf{B}_T \subseteq \mathbf{B}_R$).

EvoPath-GA Optimization (Phase 3). EvoPath-GA is a hybrid meta-heuristic operating on the pre-reduced $\mathbf{T}_{\text{reduced}}$ space. Its core innovation is the **multi-objective fitness function**: $\mathcal{F}(C) = w_{\text{cov}} \cdot f_{\text{cov}}(C) + w_{\text{div}} \cdot f_{\text{div}}(C) + w_{\mu} \cdot f_{\mu}(C) - w_{\text{cost}} \cdot f_{\text{cost}}(C)$. The algorithm incorporates heuristic seeding and a dynamic anti-cloning mechanism $\delta(g)$. Results show an **average cost reduction of 86.3**% and superior convergence compared to standard GA (Theorem 3.6).

Architecture and Operational Quality. The platform utilizes a scalable **microservices** architecture, deployed on Kubernetes and orchestrated via RabbitMQ. The **Operational Feedback Loop** (Phase 4) bridges the research-practice gap by dynamically adjusting the fitness weights w_i based on real execution cost κ_{act} and **Mutation Score** μ. This feedback mechanism achieves an average mutation score of **92.4**% (versus 90.1% for exhaustive execution), confirming qualitative superiority and meeting the **90**% quality **KPI** threshold.

This thesis validates that the combination of intelligent path reduction (SGATS)

and multi-objective optimization (EvoPath-GA) yields superior cost-quality trade-offs. KIMVIEware provides a model for translating research prototypes into industry-ready practical tools.

Keywords : Software testing, symbolic execution, genetic algorithms, microservices architecture, path explosion, test suite optimization, mutation testing, research-practice gap, SGATS, EvoPath-GA.

Resume

Contexte et Problématique. Le test logiciel constitue l'une des activités les plus coûteuses du cycle de développement moderne. Cette thèse adresse trois problèmes persistants : 1) **L'explosion des chemins d'exécution** (démontrée formellement, par exemple, 262 144 chemins pour six salles dans une simulation universitaire), 2) **La redondance des suites de tests** générées, et 3) **Le fossé académie-industrie** lié au manque de scalabilité et d'intégration CI/CD des prototypes de recherche.

Contributions Principales. Cette thèse introduit **KIMVIEware**, une plateforme générique basée sur une architecture microservices. KIMVIEware résout ces défis à travers un pipeline novateur en quatre phases intégrant exécution symbolique, réduction heuristique, optimisation génétique multi-objectifs, et retour d'information opérationnel.

Algorithme de Réduction SGATS (Phase 2). SGATS (**S**election-**G**uided **A**ggregation of **T**rajectories **S**ymbolic) est un algorithme en temps polynomial ($O(N^2 \cdot L)$) qui élimine la redondance structurelle et sémantique. Il utilise une fonction de priorité $\rho(t)$ (combinée à l'unicité et au coût) et une fonction de similarité $sim(t_i, t_j)$ (LCP et distance de Hamming). La détection de **Fusion Hit** utilise l'équivalence d'états symboliques pour la subsomption dynamique. Les résultats expérimentaux confirment un taux de réduction moyen de $\mathbf{85,1}\%$ avec une **préservation de couverture de 100**% (Théorème 3.3 : $\mathbf{B}_T \subseteq \mathbf{B}_R$).

Optimisation EvoPath-GA (Phase 3). EvoPath-GA est une méta-heuristique hybride opérant sur l'espace pré-réduit $\mathbf{T}_{\text{reduced}}$. Son innovation réside dans sa **fonction de fitness multi-objectifs** : $\mathcal{F}(C) = w_{\text{cov}} \cdot f_{\text{cov}}(C) + w_{\text{div}} \cdot f_{\text{div}}(C) + w_{\mu} \cdot f_{\mu}(C) - w_{\text{cost}} \cdot f_{\text{cost}}(C)$. L'algorithme inclut l'ensemencement heuristique et un mécanisme anti-clonage dynamique $\delta(g)$. Les résultats montrent une **réduction de coût moyenne de** $\mathbf{86,3}\%$ et une convergence supérieure par rapport au GA classique (Théorème 3.6).

Architecture et Qualité Opérationnelle. La plateforme utilise une architecture **microservices** déployée sur Kubernetes, orchestrée via RabbitMQ, garantissant modularité et scalabilité. La **Boucle de Rétroaction Opérationnelle** (Phase 4) comble le fossé académie-industrie en ajustant dynamiquement les poids de fitness w_i basés sur le coût réel κ_{act} et le **Score de Mutation** μ. Cette approche conduit à un score de mutation moyen de $\mathbf{92,4}\%$ (contre $90,1\%$ pour l'exécution exhaustive), confirmant une supériorité

qualitative et l'atteinte du seuil de 90% pour les **KPIs** de qualité.

Conclusion. Cette thèse valide que la combinaison de la réduction intelligente (SGATS) et de l'optimisation multi-objectifs (EvoPath-GA) offre des compromis coût-qualité supérieurs. KIMVIEware fournit un modèle pour la traduction des prototypes de recherche en outils pratiques prêts pour l'industrie.

Mots-clés : Test logiciel, exécution symbolique, algorithmes génétiques, architecture microservices, explosion de chemins, optimisation de suites de tests, tests de mutation, fossé académie-industrie, SGATS, EvoPath-GA.

Contents

List of Figures

List of Tables

Acronym	Meaning and Context
KIMVIEware	The proposed scalable framework for automated test case generation and optimization for microservices.
SGATS	**S**election **G**uide and **A**ggregation of **S**ymbolic **T**rajectories: The reduction procedure in Phase 2 to combat path explosion and semantic redundancy.
EvoPath-GA	**Evo**lutionary **Path** **G**enetic **A**lgorithm: The hybrid meta-heuristic optimization algorithm used in Phase 3.
]**SUT**	**S**ubject **U**nder **T**est: The compressed project or microservice package submitted for analysis.
SE	**S**ymbolic **E**xecution: The core path extraction technique in Phase 1.
CFG	**C**ontrol **F**low **G**raph: A directed graph representing the basic blocks and control transfers of the SUT.
SMT	**S**atisfiability **M**odulo **T**heories Solver: Tool used in Phase 1 to verify the feasibility (satisfiability) of a path condition.
MOOP	**M**ulti-**O**bjective **O**ptimization **P**roblem: The formal challenge solved by EvoPath-GA in Phase 3.
GA	**G**enetic **A**lgorithm: The foundational meta-heuristic upon which EvoPath-GA is built.
CI/CD	**C**ontinuous **I**ntegration / **C**ontinuous **D**eployment: Industrial pipelines that KIMVIEware is designed to integrate with.
LCP	**L**ongest **C**ommon **P**refix: A metric used in the similarity function to measure the shared initial sequence of two paths.
KPIs	**K**ey **P**erformance **I**ndicators: Metrics (like Mutation Score and Cost Reduction Ratio) tracked on the operational dashboard.
TTC	**T**ime-**t**o-**C**overage: A performance KPI measuring the time required to reach a target coverage level.
T	The initial set of N feasible execution **T**rajectories (paths) generated by Phase 1.
$\Pi(t)$	The **P**ath **C**ondition: The logical formula that must be satisfied for execution to follow trajectory t.
B_{seen}	The **B**ranches **seen**: The set of conditional branches already covered by the current paths in T_{reduced}.
$T_{\textbf{reduced}}$	The final, compact, **reduced** set of high-value **T**rajectories resulting from Phase 2.

Continued on next page

Acronym	Meaning and Context
S	The Optimal **S**et: The final, optimized test suite (subset of T_{reduced}) ready for execution.
$\mathcal{F}(C)$	The **F**itness function: The scalarized multi-objective function that EvoPath-GA seeks to maximize.
μ	**M**utation **S**core: The quality metric derived from Mutation Testing in Phase 4, representing defect detection capability[cite: 180, 185].
$\rho(t)$	**P**riority score: The score assigned to a path t by the SGATS algorithm to guide selection[cite: 75].

Chapter 1

Introduction

1.1 Context and Motivation

Software testing has long been established as one of the most critical activities in the software development lifecycle, ensuring that systems meet functional and non-functional requirements before deployment. With the rapid growth of large-scale, distributed, and data-intensive applications, the cost and complexity of testing have increased significantly. Recent studies estimate that testing activities may consume more than half of the overall development effort Prity and Hasan (2023). This high cost is further exacerbated by three persistent technical challenges: the generation of redundant test cases, the scalability limitations of symbolic execution, and the persistent misalignment between academic research and industrial practice.

One of the most widely studied challenges is the problem of **redundancy** in automated test case generation. Many approaches, particularly those based on random or evolutionary strategies, tend to produce overlapping or near-duplicate test cases that increase execution time without contributing meaningfully to coverage or fault detection Damia et al. (2021); Khan and Amjad (2015). This inefficiency reduces the practical applicability of such methods in industrial environments, where execution budgets are often tightly constrained. In parallel, symbolic execution has emerged as a powerful technique for systematically exploring program paths and generating high-coverage test inputs. However, its effectiveness is limited by the well-known **Path Explosion Problem** (PEP), in which the number of feasible execution paths grows exponentially with program size and complexity Horváth et al. (2024); Wu et al. (2024). This scalability bottleneck has restricted the use of symbolic execution in large-scale, real-world applications despite its theoretical strengths.

To address these limitations, researchers have explored a range of hybrid and optimization-based approaches. Search-Based Software Testing (**SBST**), and in particular Genetic Algorithms (GA), has shown promise in optimizing test generation by balancing coverage

and diversity while reducing redundancy Mateen et al. (2016); Damia et al. (2021). Similarly, hybrid methods that combine symbolic execution with heuristics, such as fuzzing, constraint solving improvements, or incremental exploration, have demonstrated the potential to alleviate path explosion Shuangjie Yao (2025); Wu et al. (2024). Yet, these solutions remain fragmented, often language-dependent, and rarely integrated into extensible or generic frameworks. From an architectural perspective, microservices have recently been leveraged to modularize testing platforms and to enable Testing-as-a-Service models, offering scalability and flexibility Guidi and Lanese (2019); Al-Debagy and Martinek (2018a). Nonetheless, existing microservices-based frameworks tend to be either domain-specific or experimental, limiting their broader adoption.

Beyond these technical challenges lies the more structural issue of the **academia–industry gap**. While academic research continues to produce innovative prototypes and algorithms, empirical studies reveal that very few of these tools are adopted in practice, mainly due to integration difficulties, scalability concerns, and lack of alignment with industrial priorities such as automation and cost-effectiveness Iqbal and Noor (2021); Oguz and Oguz (2019); Prity and Hasan (2023). This gap highlights the urgent need for solutions that not only push the boundaries of research but also remain accessible and useful to practitioners in real-world software engineering contexts.

Against this backdrop, the motivation of this research becomes evident. There is a pressing need for a *generic, scalable, and extensible* platform that unifies optimization techniques, symbolic execution, and modern architectural paradigms to deliver both academic value and industrial relevance. This thesis introduces **KIMVIEware**, a microservices-based platform for automated test case generation. By integrating Genetic Algorithms to reduce redundancy, hybrid symbolic strategies to mitigate path explosion, and a microservices architecture to ensure modularity and scalability, KIMVIEware aims to bridge the gap between research and practice, providing a unified and practical framework for advancing the state of software testing.

1.2 Problem Statement

Traditional test case generation approaches often suffer from inefficiency, redundancy, and poor scalability. While symbolic execution enables systematic path exploration and precise test-input synthesis, its practical applicability is severely constrained by the **Path Explosion Problem**, in which the number of feasible execution paths grows exponentially with program complexity Horváth et al. (2024); Cadar and Sen (2013). Search-based approaches, particularly Genetic Algorithms (GA), have demonstrated effectiveness in optimizing test suites by improving coverage and reducing redundancy Damia et al. (2021); Mateen et al. (2016). However, most existing GA-based solutions are applied to isolated components and are rarely integrated into generic, extensible, and industry-ready

platforms. At the same time, empirical studies report a persistent gap between academic research and industrial practice: many research prototypes do not scale or integrate poorly with CI/CD pipelines and production toolchains, which limits adoption in real-world contexts Iqbal and Noor (2021); Prity and Hasan (2023). These limitations motivate the need for a *generic, modular, and scalable* platform that composes symbolic and optimization techniques within an architecture fit for industrial deployment.

Mathematical representation of the problem

We formalize the problem to make its scale explicit. Let a program (or a microservice) P be modeled by a Control Flow Graph (CFG) $G = (N, E)$ where N is the set of program nodes (basic blocks, statements, or decision points) and $E \subseteq N \times N$ the set of control-flow edges. An execution path is a finite sequence $\pi = (n_1, n_2, \ldots, n_k)$ from an entry node n_1 to an exit node n_k, satisfying $(n_i, n_{i+1}) \in E$ for all i.

We represent branching decisions by Boolean variables: each conditional or binary decision yields a decision symbol $d_j \in \{T, F\}$. Denote by Π the set of all (syntactically) possible decision sequences (paths) in G. For simple independent constructs:

If the program contains n_b independent binary branches:

$$\Pi_{\text{branches}} = \{(d_1, \ldots, d_{n_b}) \mid d_i \in \{T, F\}\}, \quad |\Pi_{\text{branches}}| = 2^{n_b}.$$

For a loop ℓ with k_ℓ iterations and m_ℓ binary checks per iteration:

$$\Pi_{\text{loop}_\ell} = \{((d_{1,1}, \ldots, d_{1,m_\ell}), \ldots, (d_{k_\ell,1}, \ldots, d_{k_\ell,m_\ell}))\},$$
$$|\Pi_{\text{loop}_\ell}| = (2^{m_\ell})^{k_\ell}.$$

For a program composed of n_b independent branches and L loops:

$$\Pi = \Pi_{\text{branches}} \times \prod_{\ell=1}^{L} \Pi_{\text{loop}_\ell}, \qquad |\Pi| = 2^{n_b} \cdot \prod_{\ell=1}^{L} (2^{m_\ell})^{k_\ell}.$$

This compact expression demonstrates that $|\Pi|$ typically grows exponentially with the number of decisions and the number of loop iterations, rendering exhaustive exploration impractical.

Illustrative example: Timetable Management (microservice composition)

To ground the analysis in a realistic scenario closely related to this thesis, consider a *Timetable Management* system built from microservices. An end-to-end (E2E) request for course enrolment composes five collaborating microservices:

1. **AuthService(userToken)** – authentication and role check.
2. **CourseService(userId, courseId)** – course existence, prerequisites, seat reservation.
3. **RoomService(courseId, timeSlot)** – find/assign suitable room (loop over candidate rooms).
4. **ScheduleService(courseId, assignedRoom, timeSlot)** – conflict/rule checks and publish.
5. **NotificationService(userId, message)** – send email with SMS fallback.

Each service contains decisions (branches) and, in the case of RoomService, an internal loop over candidate rooms. The joint behavior of the system is the composition of the individual CFGs; hence the global set of paths approximates the Cartesian product of each service's path sets, which gives rise to combinatorial explosion when generating E2E tests.

Pseudo-algorithms (illustrative)

Below are compact pseudo-algorithms that expose the main decision points and loops (useful for extracting CFG nodes):

Algorithm 1: AuthService(userToken)

```
if not validToken(userToken) then
    return AUTH_FAIL
user ← getUser(userToken);
if user.role == "guest" then
    return PERMISSION_DENIED
return AUTH_OK, user.id
```

Algorithm 2: CourseService(userId, courseId)

```
course ← lookupCourse(courseId);
if course == NULL then
    return COURSE_NOT_FOUND
if not hasPrerequisites(userId, course.prereqs) then
    return PREREQ_NOT_MET
if course.remainingSeats <= 0 then
    return COURSE_FULL
reserveSeat(userId, courseId);
return COURSE_RESERVED
```

Algorithm 3: RoomService(courseId, timeSlot)

```
rooms ← availableRooms(timeSlot);
foreach r in rooms do
    if r.capacity ≥ course.expectedStudents and r.hasEquipment(course.needs) then
        assignRoom(r, courseId);
        return ROOM_ASSIGNED
return NO_ROOM_AVAILABLE
```

Algorithm 4: ScheduleService(courseId, assignedRoom, timeSlot)

```
if conflictsWithStudentSchedule(courseId, timeSlot) then
    return SCHEDULE_CONFLICT
if violatesDepartmentRules(courseId, timeSlot) then
    return RULE_VIOLATION
publishSchedule(courseId, assignedRoom, timeSlot);
return SCHEDULE_OK
```

Algorithm 5: NotificationService(userId, message)

```
if sendEmail(userId, message) == SUCCESS then
    return NOTIF_OK
if sendSMS(userId, message) == SUCCESS then
    return NOTIF_OK
logFailure();
return NOTIF_FAIL
```

Each conditional (‘`if`‘), loop (‘`for`‘) or fallback (‘email → SMS‘) corresponds to a branching decision or iteration that increases the path space.

Illustrative CFG (RoomService)

An illustrative (textual) CFG for `RoomService` contains:

— Entry node n_0.
— Call node n_1 computing available rooms.
— Loop head n_2 iterating over the list of candidate rooms (k iterations).
— Branch n_3: capacity check (True/False).
— Branch n_4: equipment check (True/False).
— Success exit n_5 (assign + return).
— Loop continuation n_6 (next room) and final exit n_7 (no room).

Edges: $(n_0 \to n_1), (n_1 \to n_2), (n_2 \to n_3), (n_3\_T \to n_4), (n_3\_F \to n_6), (n_4\_T \to n_5), (n_4\_F \to n_6), (n_6 \to n_2 \text{ or } n_7)$.

A graphical CFG figure may be inserted as Figure 1.1.

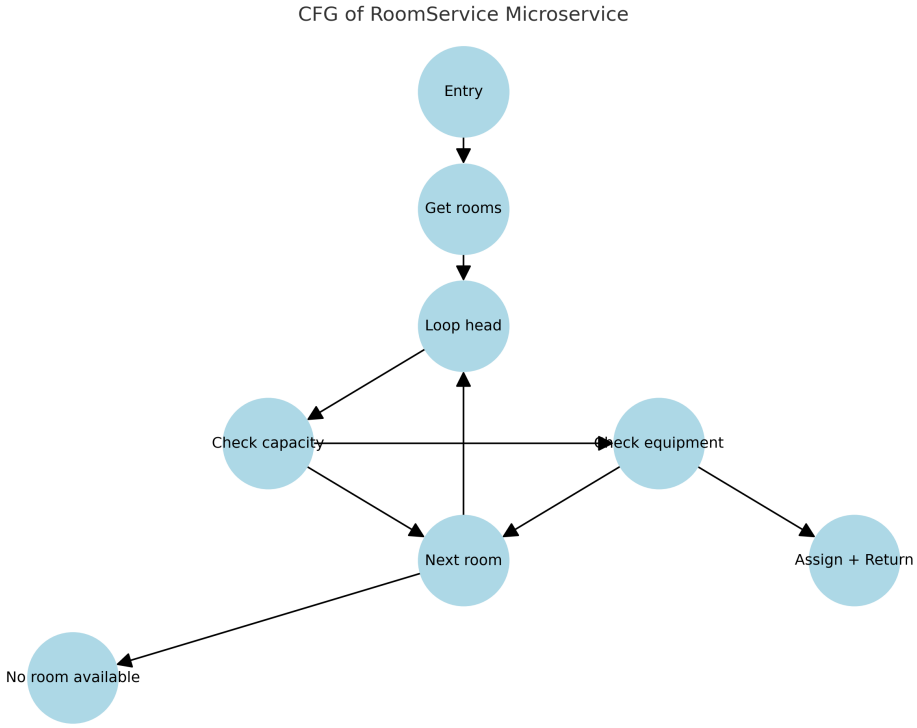

Figure 1.1 – Illustrative CFG for `RoomService` (placeholder).

Quantitative path explosion analysis

We now present numerical estimates to demonstrate the magnitude of the problem. Model each microservice S_i by the number of independent binary decisions b_i and loops with k_i iterations, where each iteration carries m_i binary checks. A conservative product approximation yields:

$$|\Pi| \approx 2^{\sum_i b_i} \cdot \prod_{i \in \mathcal{L}} (2^{m_i})^{k_i},$$

Where $\mathcal{L}$ is the set of services with loops.

Using modest, realistic values for our timetable example:

AuthService: $b_1 = 1 \Rightarrow 2^1 = 2$

CourseService: $b_2 = 3 \Rightarrow 2^3 = 8$ (Correction: 3 ifs, not 2)

RoomService: loop with $k = 3$ candidate rooms, $m = 2 \Rightarrow (2^2)^3 = 4^3 = 64$

ScheduleService: $b_4 = 2 \Rightarrow 2^2 = 4$

NotificationService: $b_5 = 1 \Rightarrow 2^1 = 2$

Hence, an approximate total number of end-to-end paths is:

$$|\Pi| \approx 2 \times 8 \times 64 \times 4 \times 2 = 8192. \quad \text{(Correction due to CourseService change)}$$

Table 1.1 shows how $|\Pi|$ increases with the number of candidate rooms k (keeping other parameters fixed).

Table 1.1 – Approximate number of end-to-end paths $|\Pi|$ for varying number of candidate rooms k (other parameters as above).

Candidate rooms k	Approx. paths $\lvert\Pi\rvert$
$k = 1$	$2 \times 8 \times 4^1 \times 4 \times 2 = 512$
$k = 3$	$2 \times 8 \times 4^3 \times 4 \times 2 = 8192$
$k = 6$	$2 \times 8 \times 4^6 \times 4 \times 2 = 524288$

The table demonstrates that even small increases in loop iterations (candidate rooms) cause explosive growth in the test space. When additional binary decisions are added to any service (for example an extra check in CourseService or ScheduleService), the total count multiplies accordingly.

Consequences for test generation and the academia–industry gap

This combinatorial blowup produces multiple operational problems when attempting automated E2E test generation for microservices:

— **Constraint composition complexity:** Generating coherent inputs that satisfy cross-service constraints (e.g., a valid token, an existing course with seats, and a compatible time slot with an available room) often requires solving large, multi-component constraint sets (distributed SMT problems). This is far heavier than single-service symbolic solving and multiplies the number of SMT queries (costly in time and resources).

— **Execution cost:** Running E2E tests implies deploying or simulating multiple services, databases, and external dependencies—each test is significantly more expensive than unit tests.

— **Redundancy and wasted budget:** Many generated paths are near-duplicates in terms of coverage or fault-detection potential; executing them wastes compute time and delays feedback to developers.

— **Non-determinism and flaky behaviour:** Network timeouts, retries and asynchronous message passing introduce additional branches that depend on timing, increasing state space and producing flakiness which complicates result interpretation.

— **Mismatch with industrial constraints:** Academic prototypes often target small benchmarks or single services; in industry, the need is for scalable, CI-friendly tooling that respects time budgets, resource quotas, and integrates with existing pipelines Prity and Hasan (2023); Iqbal and Noor (2021).

These observations explain why symbolic execution and SBST techniques perform well on academic benchmarks but struggle when confronted with integrated multi-service architectures. A practical testing platform must therefore (1) extract and compose constraints in a distributed manner, (2) reduce the search space intelligently (prioritization/merging), (3) optimize selection of compact, high-value test suites (e.g., via GA), and (4) provide scalable orchestration for execution and trace collection. These are precisely the problems addressed by KIMVIEware in this thesis.

1.3 Research Aim and Objectives

Research Aim

The overarching aim of this research is to design and develop **KIMVIEware**, a generic and extensible platform based on microservices architecture, which enables automated test case generation by combining symbolic execution and optimization strategies. The platform aims to mitigate scalability challenges, such as path explosion, and reduce redundancy in test suites, while also bridging the gap between academic research and industrial practice by providing a reusable and industry-ready solution.

Research Objectives

To achieve this aim, the research pursues the following specific objectives:

— **Architectural design:** To design and implement a microservices-based architecture that modularizes the different testing components and supports scalability, reusability, and extensibility.
— **Optimization integration:** To incorporate genetic algorithms optimized for test case generation and selection, aiming to maximize coverage while minimizing redundancy.
— **Path explosion mitigation:** To propose and evaluate hybrid strategies combining symbolic execution, path prioritization, and constraint solving to address the exponential growth of execution paths.
— **Scalability and adaptability:** To employ software engineering design patterns and best practices that ensure the platform can evolve and integrate with diverse testing contexts.

— **Practical relevance:** To validate the platform through benchmarks and case studies that demonstrate its effectiveness and its potential adoption in industrial settings.

1.4 Research Questions

The following research questions guide this thesis:

1. **RQ1:** How can genetic algorithms be effectively applied to optimize test case generation and selection while reducing redundancy?
2. **RQ2:** What hybrid strategies can be designed to mitigate the path explosion problem in symbolic execution for large-scale software?
3. **RQ3:** How can a microservices-based architecture support scalability, modularity, and extensibility in automated software testing platforms?
4. **RQ4:** To what extent can a generic platform reduce the gap between academic research and industrial practices in software testing?

1.5 Significance of the Study

The significance of this study can be considered from three complementary perspectives: academic contribution, industrial relevance, and knowledge transfer.

Academic Contribution

This research advances the body of knowledge in software testing by proposing a novel integration of three complementary paradigms:

— **Search-based optimization:** The use of genetic algorithms to optimize test case generation and selection contributes to the growing field of Search-Based Software Testing (SBST).
— **Symbolic execution:** The combination of symbolic analysis with hybrid heuristics directly addresses the well-documented path explosion problem in complex programs.
— **Software architecture:** The adoption of microservices as the backbone of a testing platform represents a methodological innovation, offering modularity and reusability for future research.

Through this integration, the study demonstrates how techniques traditionally explored in isolation can be unified into a coherent and extensible framework.

Industrial Relevance

From an industrial perspective, the proposed KIMVIEware platform is significant because it:

— Provides a scalable and cost-effective approach to automated testing, which can reduce the overall testing effort and accelerate release cycles.
— Offers modular components that can be integrated into modern CI/CD pipelines, thereby improving the feasibility of adoption in practice.
— Enhances defect detection by systematically combining symbolic and search-based approaches, reducing the risk of undetected failures in deployed systems.

In doing so, the platform addresses critical concerns of scalability, redundancy elimination, and practical automation that are at the forefront of industrial testing challenges.

Knowledge Transfer and Societal Impact

Ultimately, the research helps bridge the gap between academic research and industrial practice. By producing a generic, reusable, and extensible platform, this study supports:

— **Knowledge transfer:** Translating advanced academic concepts into tools that practitioners can adopt with minimal adaptation.
— **Reproducibility:** Providing a reference architecture and experimental framework that future researchers can replicate, extend, or benchmark against.
— **Societal impact:** Enhancing software quality and reliability in domains such as education, healthcare, and enterprise systems, where failures may have significant economic or human consequences.

Overall, the significance of this work lies in its dual role: it contributes to advancing academic knowledge while delivering practical value to industry. In doing so, it positions KIMVIEware as a stepping stone toward the next generation of automated, scalable, and industry-ready software testing platforms.

1.6 Scope of the Study

The scope of this research is intentionally delimited to ensure focus, feasibility, and clarity of contribution. It defines both the boundaries of investigation and the aspects that are excluded from consideration.

In-scope elements

This study specifically focuses on:

— **Microservices architecture:** The design and implementation of a modular, microservices-based platform (KIMVIEware) that orchestrates automated test generation components.
— **Automated test case generation:** The integration of symbolic execution and genetic algorithms to produce efficient, high-coverage test cases while minimizing redundancy.
— **Path explosion mitigation:** The exploration of hybrid strategies that combine constraint solving, path prioritization, and search-based heuristics to address scalability limitations.
— **Evaluation:** Benchmarking the proposed platform using realistic academic service scenarios such as *student grade management* and *timetable scheduling*, which reflect distributed microservice-based workflows. These case studies are chosen because they are representative, reproducible, and sufficiently complex to expose scalability challenges.

By clearly delimiting its scope, the study ensures that contributions are concentrated on advancing automated test case generation in microservices environments. The benchmarks selected (university-oriented services such as grade management and timetable scheduling) provide a relevant and controlled context for experimentation, while leaving broader domains and specialized testing activities to future research.

1.7 Thesis Structure

The remainder of this thesis is organized into four main subsequent chapters. **Chapter 2** (2) first reviews the relevant literature, establishing the state of the art in automated software testing, symbolic execution, Search-Based Software Testing (SBST), and microservices-based platforms, which critically highlights the existing research gap. **Chapter 3** (3) then presents the detailed research methodology and the architectural design of the **KIMVIEware** platform, formally defining the SGATS Reduction Algorithm and the EvoPath-GA Optimization Model. Following this, **Chapter 4** (4) details the technical implementation, the experimental setup, and the quantitative evaluation, presenting and analyzing the results obtained from the realistic microservice case studies. Finally, **Chapter 5** (5) concludes the thesis by summarizing the key research outcomes, discussing the findings relative to the research questions, highlighting the academic and industrial contributions, and outlining directions for future work and limitations.

Chapter 2

State of the Art

2.1 Definitions of Key Concepts

2.1.1 Software Testing

Software testing is the process of executing software artifacts to detect defects and ensure compliance with functional and non-functional requirements Myers et al. (2011); Beizer (1995). Two broad categories are generally distinguished:

— **Structural testing (white-box):** techniques that exploit the internal structure of the program to design tests aiming for maximum coverage of code elements such as statements, branches, or paths. While effective in exposing hidden logic errors, these approaches are often resource-intensive and do not guarantee the detection of missing functionalities.

— **Functional testing (black-box):** methods that rely on the system's functional specification without referencing its source code. Input–output relations are derived from requirements, which makes this strategy particularly suitable for testing compilers, circuit simulators, and embedded systems where correctness depends on compliance with formal specifications Beizer (1995).

In functional testing, grammars (context-free, attribute-based, or parametric) have historically been employed to represent input syntax and systematically generate test data. Such techniques have been applied to diverse domains, including compiler validation, VLSI circuit simulation, graphical user interface software, sorting and merging programs, communication protocols, and operating systems Beizer (1995).

Several criteria guide the design of an effective test generator: (1) maximizing code or specification coverage, (2) controlling the computational cost of generation and execution, (3) enabling prediction and verification of expected results, (4) targeting different classes of errors, (5) providing partial exhaustiveness where full coverage is infeasible, and (6) maintaining usability and reproducibility.

Automatic test generation offers several advantages: the rapid production of large

test suites, the diversity of combinations, the reproducibility of randomized tests, and significant time savings compared to manual design. However, limitations persist. It is often unclear how many tests are sufficient; predicting expected outputs is difficult in complex systems (the oracle problem), extreme cases are not always well-managed, and such methods are less effective for simple or highly interactive software.

In conclusion, automatic test generation should be regarded as a *complement* to classical testing techniques rather than a full replacement. Ongoing research explores ways to strengthen its effectiveness, for example by leveraging formal specifications or hybrid symbolic–search approaches to improve both generation and verification McMinn (2004); Prity and Hasan (2023).

2.1.2 Test Case Generation

Test case generation is the process of systematically creating inputs and expected outputs to evaluate a program under test (PUT). The aim is to maximize coverage and fault detection while minimizing redundancy and computational cost McMinn (2004); Prity and Hasan (2023). The general process can be decomposed into five steps:

1. **Input domain identification:** determining the valid ranges and types of inputs, along with any preconditions or invariants that constrain them.
2. **Constraint modeling:** capturing rules that inputs must satisfy, either from specifications or inferred from code.
3. **Test generation strategy:** producing test inputs through random testing, combinatorial methods, symbolic execution, or search-based techniques.
4. **Execution of test cases:** running the generated inputs on the PUT to observe outputs and system behavior.
5. **Adequacy evaluation:** measuring the effectiveness of the generated tests using criteria such as statement, branch, or path coverage, as well as mutation score or fault-detection rate.

The Oracle Problem. A critical challenge in test case generation is the *oracle problem*, which refers to the difficulty of determining whether the observed outputs of the PUT are correct Liu et al. (2014). The problem arises in several forms:

— **Absence of reference outputs:** in domains such as numerical computation, artificial intelligence, or adaptive systems, there may be no single correct output. For example, a recommender system produces personalized suggestions without a definitive ground truth Mao et al. (2024).

— **High cost of constructing an oracle:** in scientific simulations or engineering domains, precise oracles exist but require computationally expensive high-fidelity models or expert judgment.

— **Trustworthiness of the oracle:** In safety-critical domains, such as avionics or medical software, the oracle itself must be verified, which reduces its usability as a lightweight mechanism.

Strategies to Address the Oracle Problem. Several approaches mitigate this limitation:

— **Metamorphic Testing (MT):** leverages metamorphic relations (MRs) that express properties between multiple executions with related inputs, enabling consistency checking without explicit expected outputs Chen et al. (2020); Mao et al. (2024).

— **Mutation Testing:** introduces small syntactic changes (mutants) into the code and evaluates whether the test suite detects them, using mutant-killing as a proxy for adequacy Papadakis et al. (2019).

— **Approximate Oracles:** accept outputs within tolerance thresholds, particularly relevant in floating-point or stochastic computations Kanewala and Bieman (2014).

— **Statistical Oracles:** analyze output distributions rather than individual results, using hypothesis testing to detect anomalies in randomized or probabilistic systems Guderlei and Mayer (2007).

Summary. Test case generation is a cornerstone of automated testing; however, the oracle problem limits its practical impact. Advances in symbolic execution and search-based testing have improved input diversity and coverage, yet fault detection remains limited without reliable oracles. This motivates hybrid approaches that combine automated generation with complementary validation mechanisms such as metamorphic or mutation testing, paving the way for more scalable and industry-relevant solutions.

2.1.3 Genetic Algorithms (GA)

Genetic Algorithms (GA) are a class of evolutionary optimization techniques inspired by Darwinian principles of natural selection Holland (1992). They operate on a population of candidate solutions, evolving them through iterative application of genetic operators. In software testing, GA are widely applied to optimize test case generation by maximizing coverage, minimizing redundancy, and targeting critical execution paths McMinn (2011); Rajagopal and Kumar (2023).

The standard GA process follows six stages:

1. **Initialization:** an initial population of candidate test cases is generated, either randomly or seeded from existing tests. For microservices, each candidate may represent an input sequence such as `(login, addCourse, generateTimetable)`.

2. **Fitness evaluation:** each test case is evaluated using a fitness function. Standard criteria include statement or branch coverage, path diversity, fault-detection potential, or mutation score. For instance, a test that exercises rare service interactions receives higher fitness.
3. **Selection:** fitter candidates are more likely to be chosen as parents for the next generation. Techniques include roulette-wheel selection, tournament selection, or rank-based strategies.
4. **Crossover (recombination):** two parent test cases are combined to form offspring. For example, the prefix of one input sequence may be combined with the suffix of another, producing new test flows across services.
5. **Mutation:** random modifications are applied to offspring to introduce diversity and prevent premature convergence. In test generation, mutation may involve altering parameter values, inserting additional API calls, or modifying service request order.
6. **Termination:** the process continues until a stopping condition is met, such as reaching a maximum number of generations, convergence of fitness values, or achieving a target coverage threshold.

Example in Test Case Generation. Consider a university microservice platform with services such as `AuthService`, `CourseService`, and `TimetableService`. A GA-based generator may begin with random API call sequences. Through successive generations, the GA evolves these sequences to maximize path coverage across service interactions. For instance, initial candidates may redundantly test only `AuthService`, but crossover and mutation operations can produce richer sequences like `login → addCourse → generateTimetable → removeCourse`, thereby covering multi-service workflows.

Strengths and Limitations. GA provides a powerful mechanism to explore significant and complex input spaces, efficiently generating high-coverage test suites. Their stochastic nature ensures diversity and robustness against local optima. However, they require careful design of fitness functions, as they can produce redundant tests, and may demand significant computational resources for convergence. In practice, GA are most effective when integrated with complementary techniques such as symbolic execution or path prioritization, as explored in this thesis.

2.1.4 Symbolic Execution

Symbolic execution is a program analysis technique that explores program behavior by replacing concrete inputs with symbolic variables and propagating symbolic expressions during execution. Instead of executing with specific values, the program is executed with

symbols representing arbitrary values, and constraints over these symbols are accumulated to capture feasible program paths King (1976); Cadar and Sen (2013).

At each conditional branch, symbolic execution explores both outcomes by updating the symbolic state. Let a symbolic state be represented as a tuple $s = \langle \sigma, pc, \pi \rangle$, where: - σ is the symbolic store (mapping program variables to symbolic expressions), - pc is the program counter, and - π is the accumulated path condition.

For a branch predicate b encountered at instruction i, the state s forks into two successor states:

$$s_{\text{then}} = \langle \sigma, pc + 1, \pi \wedge b \rangle, \qquad s_{\text{else}} = \langle \sigma, pc', \pi \wedge \neg b \rangle \tag{2.1}$$

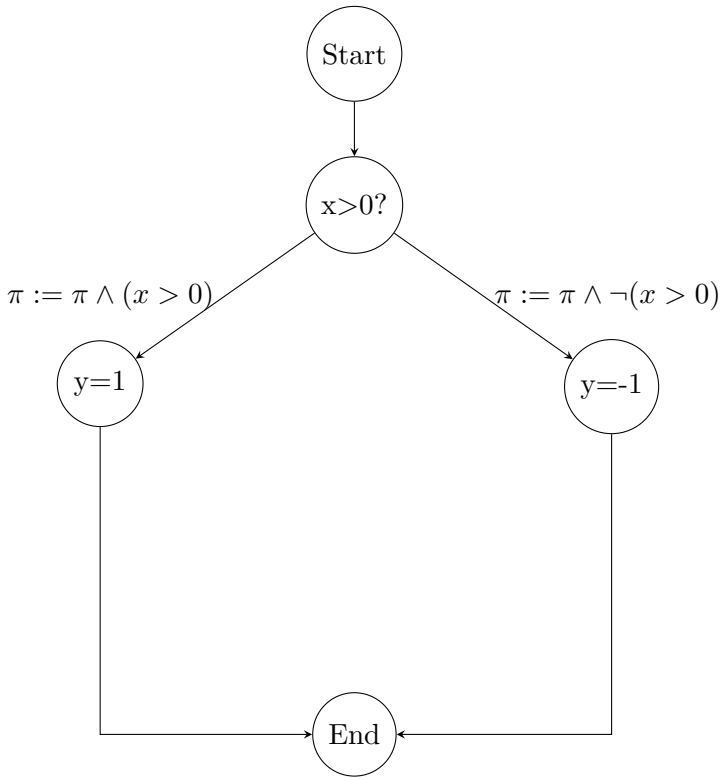

Figure 2.1 – Illustration of symbolic execution branching at a conditional statement. Each successor state refines the path condition π depending on the branch outcome.

where $pc + 1$ is the instruction following the `then`-branch, and pc' is the instruction at the `else`-branch.

Only states with satisfiable path conditions π are retained, as determined by an SMT solver (e.g., Z3, STP). This branching semantics naturally induces an *execution tree*, where each leaf corresponds to a feasible path characterized by its path condition.

This semantics naturally induces an *execution tree*, where each path is annotated with a path condition π. Each leaf corresponds to a conjunction of constraints that inputs must satisfy to traverse that path Cadar and Sen (2013).

Core Components. Symbolic execution maintains three fundamental elements throughout exploration Cadar and Sen (2013):

— **Path condition (π):** the conjunction of predicates collected along the current execution path.

— **Program counter:** the location of the next instruction to be executed.

— **Symbolic state:** symbolic expressions representing program variables and memory contents.

These elements allow symbolic execution engines to systematically enumerate program behaviors and reason about infeasible or equivalent paths.

Constraint Solvers. The effectiveness of symbolic execution critically depends on the constraint solvers used to reason about path conditions π. These solvers are invoked to: (a) check the feasibility of path conditions during exploration, and (b) generate concrete inputs that satisfy feasible π.

Early symbolic execution frameworks relied on **STP**, an efficient solver for bit-vectors and arrays, which was widely adopted in tools like KLEE Cadar and Sen (2013). More recently, **Z3**, developed by de Moura and Bjørner, has become the most widely used SMT solver due to its support for a broad range of background theories (linear arithmetic, bit-vectors, arrays, quantifiers) and its balance between scalability and precision de Moura and Bjørner (2008).

Several symbolic execution engines build upon or integrate with these solvers:

— **angr**: a binary analysis platform that leverages Z3 to perform path exploration and vulnerability detection in binaries without source code Shoshitaishvili et al. (2016).

— **Java PathFinder (JPF)**: an explicit-state model checker extended with symbolic execution capabilities, using specialized constraint solvers for Java bytecode programs Visser et al. (2003).

— **JBSE**: the Java Bytecode Symbolic Execution engine, designed specifically for automated test generation in Java, integrating solvers like Z3 for path feasibility checking Frisiani and Riganelli (2017).

These tools highlight the interplay between symbolic execution and constraint solving: the choice of solver directly impacts scalability, supported domains, and ultimately the applicability of symbolic execution to real-world systems (e.g., hardware verification, embedded software, or security analysis) Ryan and Sturton (2023). However, despite significant progress, constraint solving remains a major bottleneck for large-scale or highly complex systems.

Strengths and Applications. Symbolic execution provides systematic exploration with strong guarantees on path coverage. It has been successfully applied in compiler

validation, operating system testing, malware analysis, and verification of safety-critical software Cadar and Sen (2013); Sen et al. (2005).

Limitations. Despite its power, symbolic execution faces several well-known limitations:

— **Path explosion:** the number of feasible paths grows exponentially with program size, especially in the presence of loops and nested branches Cadar and Sen (2013); Horváth et al. (2024).

— **Solver bottlenecks:** constraint solving becomes computationally expensive when handling complex constraints or large input domains Ryan and Sturton (2023).

— **State space explosion:** symbolic execution of memory-intensive programs produces large symbolic states, limiting scalability Cadar and Sen (2013).

These challenges motivate hybrid approaches where symbolic execution is combined with heuristics such as fuzzing or search-based algorithms (e.g., genetic algorithms), which is one of the central strategies adopted in this thesis.

2.1.5 Microservices Architecture

Microservices architecture decomposes applications into loosely coupled, independently deployable services that communicate through lightweight protocols, such as REST or message queues Newman (2015). Key characteristics include:

— **Independence:** services can be developed, deployed, and scaled independently.

— **Granularity:** each service encapsulates a specific business capability.

— **Communication:** services interact via APIs, often in distributed and heterogeneous environments.

— **Orchestration:** service coordination is managed through tools such as Kubernetes or service meshes.

For testing, microservices introduce new opportunities (e.g., service-level isolation, parallel testing) but also challenges such as *service interaction complexity*, distributed states, and fault propagation Soldani et al. (2018); Al-Debagy and Martinek (2018b).

2.1.6 Academia–Industry Gap in Software Testing

The *academia–industry gap* refers to the persistent mismatch between the innovations produced by academic research and the practical needs of the software industry Iqbal and Noor (2021); Oguz and Oguz (2019). While universities and research labs frequently propose novel techniques—such as symbolic execution engines, optimization-based test generators, and automated verification frameworks—empirical studies consistently report that very few of these tools are adopted in industrial practice.

Evidence of the Gap. A recent mapping study reveals that, despite decades of research in automated testing, fewer than 10% of academic prototypes achieve sustained use in industry Prity and Hasan (2023). Similarly, interviews with practitioners reveal a reliance on traditional unit testing frameworks (e.g., JUnit, pytest) and manual test design, while advanced academic tools are rarely integrated into industrial toolchains Ali and Khan (2020); Iqbal and Noor (2021).

Barriers to Adoption. The main reasons behind this gap can be grouped into four categories:

— **Scalability:** academic prototypes often fail to handle the size and complexity of real-world systems, particularly distributed microservice architectures.

— **Integration:** industrial environments rely heavily on CI/CD pipelines, containerization, and orchestration tools. Academic tools are seldom designed to integrate seamlessly with these ecosystems.

— **Usability and support:** many research tools lack documentation, long-term maintenance, or community adoption, making them impractical for enterprise use.

— **Cost and priorities:** companies often prioritize speed, automation, and cost-effectiveness over exhaustive verification, while academic work emphasizes theoretical completeness.

Implications. This misalignment results in a paradox: while academic research continues to advance state-of-the-art techniques, practitioners remain constrained to traditional, less efficient methods. Bridging this gap requires solutions that combine academic rigor with industrial feasibility, emphasizing modularity, extensibility, and ease of deployment. The proposed **KIMVIEware** platform is designed with this dual objective in mind: to serve as both a research framework and a practical tool for industry adoption.

2.1.7 Design Patterns

Design Patterns represent generic, reusable, and proven solutions to recurring software engineering problems. Their formalization was established as a standard in the landmark book **Design Patterns: Elements of Reusable Object-Oriented Software** (1994), authored by the « Gang of Four » (**GoF**) Gamma et al. (1994). The systematic adoption of these patterns is essential for ensuring **modularity**, **extensibility**, and **maintainability**—critical goals for any modern automated testing platform. Design patterns are categorized into three prominent families, each contributing to the system's architectural flexibility.

Creational Patterns (**Creational Patterns**) manage object instantiation mechanisms, providing ways to create objects flexibly and independently, hiding the construction logic from the client. These patterns, including **Factory**, **Builder**, and **Singleton**, are

crucial for isolating the complexity of component initialization. For instance, the Factory pattern can be used to generate different types of analysis modules (such as parsers or input generators) without exposing the underlying instantiation logic, thus ensuring maximum decoupling. The Singleton pattern guaranteensures that a critical class (such as a global configuration manager or a resource pool) has only a single instance, thereby guaranteeingency and controlled resource sharing in a distributed microservices architecture.

Structural Patterns (**Structural Patterns**) focus on how classes and objects are composed to form larger, cohesive structures. They are of paramount importance in microservices architectures. The Facade pattern simplifies the complex interface of a subsystem (e.g., an interface to a static analysis library) for other modules, enhancing usability. The Adapter pattern allows incompatible interfaces (such as different formats of analysis tools or SMT solvers) to collaborate seamlessly. The Bridge pattern is compelling in this context, as it allows an abstraction to be decoupled from its implementation, enabling both parts to evolve independently. This decoupling is vital for maintaining flexibility between different analysis modules and the third-party tools they encapsulate.

Behavioral Patterns (**Behavioral Patterns**) define the communication schemes and the assignment of responsibilities among objects. Patterns like **Observer** and **Command** are crucial for implementing asynchronous logic and monitoring systems. The Observer pattern is central to notification mechanisms, allowing components to automatically inform dependencies (such as a reporting or monitoring service) of any state change. The Strategy pattern is highly relevant in testing, as it allows for the dynamic selection of an algorithm or behavior at runtime, offering the flexibility to switch between different test prioritization strategies or various search algorithm implementations (such as Genetic Algorithms and Simulated Annealing) without requiring architectural modifications to the framework.

2.2 Software Testing and its Challenges

Having defined the key concepts, we now turn to the main challenges that motivate this thesis. Software testing represents one of the most resource-intensive activities in the software development lifecycle, often consuming between 30% and 50% of project budgets Prity and Hasan (2023). While indispensable for ensuring correctness, safety, and reliability, modern systems introduce new difficulties that make testing increasingly complex.

The problem of **redundancy** arises when testing techniques generate excessive or overlapping test cases. Redundant tests inflate execution time and resource consumption without yielding proportional improvements in coverage or fault detection. For example, many automated techniques, including evolutionary test generation, struggle to avoid producing similar test cases that contribute little additional value.

Scalability is another pressing issue. Large-scale software systems, especially those with millions of potential execution paths, expose the limitations of conventional techniques. Symbolic execution, for instance, is known to encounter exponential blow-up in the number of explored paths, making it impractical for real-world applications without additional mitigation Horváth et al. (2024). Similarly, search-based testing must contend with very large search spaces, which challenge the efficiency of heuristic optimization methods.

A third challenge concerns **realism**. As Wu et al. Wu et al. (2024) note, synthetic inputs often fail to represent actual usage scenarios. Unrealistic test inputs can produce high coverage in theory but miss faults that only emerge under realistic data distributions. In domains such as Big Data systems or safety-critical software, realism in testing is not a luxury but a necessity for uncovering hidden defects.

Finally, a well-documented **practicality gap** separates academic research and industrial practice. Iqbal et al. Iqbal and Noor (2021) emphasize that despite a wealth of sophisticated research prototypes, industry adoption remains limited. The reasons include poor scalability, lack of integration with existing toolchains, and limited alignment with industrial priorities such as automation and cost reduction. aiming to reducechallenges underscore the need for advanced approaches that combine the strengths of multiple paradigms—such as search-based optimization, symbolic execution, and scalable architectures—into unified solutions. This thesis contributes to that objective by proposing KIMVIEware, a generic microservices-based platform that integrates these techniques.

2.3 Automated Test Case Generation

The automation of test case generation has been a central focus of research, aiming to reduce human effort while enhancing fault detection and coverage. Early approaches relied heavily on random or heuristic-based methods, but more sophisticated techniques have emerged under the umbrella of Search-Based Software Testing (SBST).

2.3.1 Search-Based Software Testing (SBST)

SBST conceptualizes test case generation as an optimization problem, where the goal is to search for input data that maximizes coverage or fault detection. Metaheuristic algorithms such as simulated annealing, tabu search, and evolutionary algorithms have been extensively applied Mateen et al. (2016). Their strength lies in balancing exploration of the search space with exploitation of promising solutions, thus enabling test generation even in large and complex domains. However, SBST methods are sensitive to parameter tuning and may suffer from local optima, making them computationally expensive for large-scale applications.

2.3.2 Genetic Algorithms in Test Case Generation

Among SBST methods, Genetic Algorithms (GA) have proven particularly effective. Inspired by natural selection, GAs maintain populations of candidate test cases that evolve over generations according to fitness functions such as branch coverage, mutation score, or fault detection capability. Operators such as crossover and mutation introduce diversity, while selection mechanisms ensure that high-quality test cases propagate.

Empirical studies demonstrate the usefulness of GAs in various testing contexts. For example, Damia et al. Damia et al. (2021) proposed the Enhanced Multiple-Searching Genetic Algorithm (EMSGA) to address redundancy and improve exploration in test case generation. Similarly, Khan et al. Khan and Amjad (2015) showed how GAs can optimize test suites by reducing redundancy while maintaining coverage. Multi-objective GAs have also been employed to balance trade-offs such as minimizing suite size and maximizing fault detection Author and Author (2020).

Despite their effectiveness, GA-based methods have limitations. They are computationally expensive, often tailored to specific programming languages or domains, and rarely integrated into extensible platforms. These shortcomings highlight the need for more generic, reusable, and scalable solutions, such as those envisioned in KIMVIEware.

2.4 Symbolic Execution and the Path Explosion Problem

Symbolic execution has long been recognized as a powerful technique for systematically exploring program paths. By treating inputs as symbolic variables and using constraint solvers to determine feasible paths, symbolic execution achieves high coverage and generates precise test cases. However, its promise is significantly constrained by the **path explosion problem**, where the number of execution paths grows exponentially with program size and complexity Horváth et al. (2024).

Mitigation Strategies Numerous strategies have been proposed to mitigate path explosion. One approach is **path prioritization**, where only a subset of paths the deployment of modular microservices or fault detection are explored Shuangjie Yao (2025). Another direction is to improve **constraint solving**, as solvers like Z3 de Moura and Bjørner (2008) often constitute the bottleneck in symbolic execution. Hybrid approaches, which combine symbolic execution with heuristic methods such as fuzzing or evolutionary search, have also shown promise in reducing the combinatorial explosion Wu et al. (2024); Author and Author (2023).

Nevertheless, no single method fully eliminates path explosion. Existing solutions often trade precision for scalability, or are constrained to specific programming languages.

This persistent limitation makes path explosion one of the central challenges addressed in this thesis, where hybridization with genetic algorithms and modular microservices deployment are proposed as promising avenues.

2.5 Microservices Architecture in Software Testing

Microservices architectures have revolutionized software engineering by promoting modularity, scalability, and independent deployment. Instead of building monolithic systems, applications are decomposed into loosely coupled services that communicate through lightweight protocols. In the context of testing, microservices architectures offer unique advantages, including reusability of components, on-demand scalability, and the ability to integrate heterogeneous testing techniques Al-Debagy and Martinek (2018a).

Microservices for Testing Platforms Applying microservices principles to testing platforms has been explored in recent works. Guidi and Lanese Guidi and Lanese (2019) developed a testing framework using the Jolie programming language, demonstrating how orchestration of test execution across services can be automated. Their work highlights the benefits of modularity but is constrained by language specificity. More generally, microservices enable the creation of Testing-as-a-Service solutions, where test generation and execution are dynamically provisioned. Design patterns, such as Adapter, Facade, and Proxy, enhance integration and modularity, ensuring that platforms can scale to meet industrial needs.

These architectural benefits make microservices an attractive foundation for KIMVIEware, allowing it to integrate search-based testing and symbolic execution into a unified, scalable platform.

2.6 Bridging the Academia–Industry Gap

Despite substantial progress in testing research, a persistent gap remains between academic advances and industrial adoption. Studies show that while academia prioritizes innovation and theoretical rigor, industry emphasizes practicality, automation, and cost efficiency Prity and Hasan (2023); Iqbal and Noor (2021). This misalignment results in limited adoption of academic tools and methods in practice.

Several factors contribute to this gap. Prity and Hasan Prity and Hasan (2023) highlight that industrial practitioners often find academic tools difficult to integrate with existing workflows. In contrast, Oguz et al. Oguz and Oguz (2019) note that many companies lack the expertise to adopt advanced methods such as symbolic execution. Furthermore, educational curricula often underrepresent testing and automation, leaving graduates unprepared for industry needs Ali and Khan (2020).

Bridging this gap requires not only technical innovation but also careful attention to usability, scalability, and integration with industry toolchains. This thesis addresses this challenge directly by designing KIMVIEware as both a research contribution and a practical solution aligned with industrial practices.

2.7 Current testing methodology and its limitations

The literature review highlights significant advances in software testing research, particularly in the areas of test case generation, symbolic execution, and search-based optimization. Genetic Algorithms (GA) have proven effective in exploring large input spaces and reducing redundancy, while Symbolic Execution (SE) provides systematic coverage and precise test input generation. Microservices architectures, meanwhile, offer modularity, scalability, and extensibility, making them an attractive foundation for testing platforms. However, each of these approaches faces persistent challenges: GA methods remain computationally intensive and prone to redundancy, SE suffers from the path explosion problem, and microservices-based testing frameworks are often language-specific or experimental.

Beyond these technical challenges lies the structural **gap between academic research and industrial practice**. Despite decades of research, the adoption of advanced testing techniques in industry remains limited due to scalability constraints, integration difficulties, and misalignment with industrial priorities.

These observations define the research gap that motivates this thesis: the absence of a *generic, scalable, and extensible platform* that simultaneously integrates (i) optimization-based strategies such as GA, (ii) systematic exploration via SE, (iii) a microservices architecture for modularity and scalability, and (iv) a design explicitly oriented toward bridging the academia–industry divide.

In response, this thesis introduces **KIMVIEware**, a unified microservices-based testing platform that leverages GA and SE to address redundancy and path explosion, while emphasizing reusability, scalability, and practical relevance for both academic and industrial stakeholders.

Table 2.1 – Critical Synthesis of Selected Studies Relevant to Search-Based and Symbolic Testing

Authors	**Problem Addressed**	**Methodology / Solution**	**Datasets / Evaluation**	**Limitations**	**Link with Thesis (KIMVIEware)**
Damia et al. (2021)	**Redundant / inefficient test cases** via standard GA.	**EMSGA** (GA with chromosome selection for size reduction).	EvoSuite benchmarks (SF110 Java). Coverage metrics.	High complexity; **language-specific (Java)**.	Inspires GA module; focus on **diversity/path reduction**.
Horváth et al. (2024)	**Path explosion** in large-scale Symbolic Execution (SE).	Incremental SE with Clang/CodeChecker. **Dynamic static analysis** to prune paths.	Industrial C/C++ projects.	**Limited to C-family languages**.	Tactics for **mitigating path explosion** (**SGATS Reduction Algorithm**).
Wu et al. (2024)	**Unrealistic test inputs** in Big Data testing.	**Naturalness-aware SE**. Incorporates distributional constraints into SMT solving.	DISC benchmarks. Test input quality focus.	**Requires manual annotations**; domain-limited.	Demonstrates injecting **domain-specific constraints** into SE.
Guidi & Lanese (2019)	Lack of robust **orchestration frameworks** for microservice testing.	Jolie-based testing framework. Formal orchestration of test execution.	Case studies using Jolie language.	**Dependence on the Jolie language**.	**Motivates generic, language-agnostic microservices architecture**.
Prity & Hasan (2023)	Persistent **academia–industry gap** in advanced tool adoption.	Qualitative studies (interviews) on gaps in practices/skills.	Empirical data from 11 regional practitioners.	Small sample, limiting **generalizability**.	**Justifies industry-oriented, modular, containerized design**.
Yao & She (2025)	Ineffective prioritization of numerous feasible paths in SE.	**EMP-C** (Effective Path Prioritization). Lightweight path cover metric.	Standard public benchmarks (Siemens Suite).	Performance may degrade on highly stateful microservices.	Confirms **criticality of path prioritization** for **SGATS Reduction Algorithm**.

Chapter 3

Methodology and System Design

This chapter presents the methodological foundation and design of the proposed platform **KIMVIEware**, a scalable framework for automated generation and optimization of test cases for microservices. The approach is organized as a four-phase workflow that extracts program paths, reduces redundancy, optimizes with an evolutionary algorithm, and produces a compact, high-coverage test suite. Each phase is supported by rigorous formal definitions, algorithms, and design principles to guarantee automation, modularity, scalability, and traceability.

3.1 Global Workflow

The proposed methodology is structured around a four-phase workflow that integrates symbolic execution, réduction techniques, genetic optimization, and microservices-based orchestration into a unified testing platform called **KIMVIEware**. Figure 3.1 illustrates the end-to-end workflow, where raw program paths are systematically transformed into optimized and executable test cases, monitored in real-time through a dedicated dashboard.

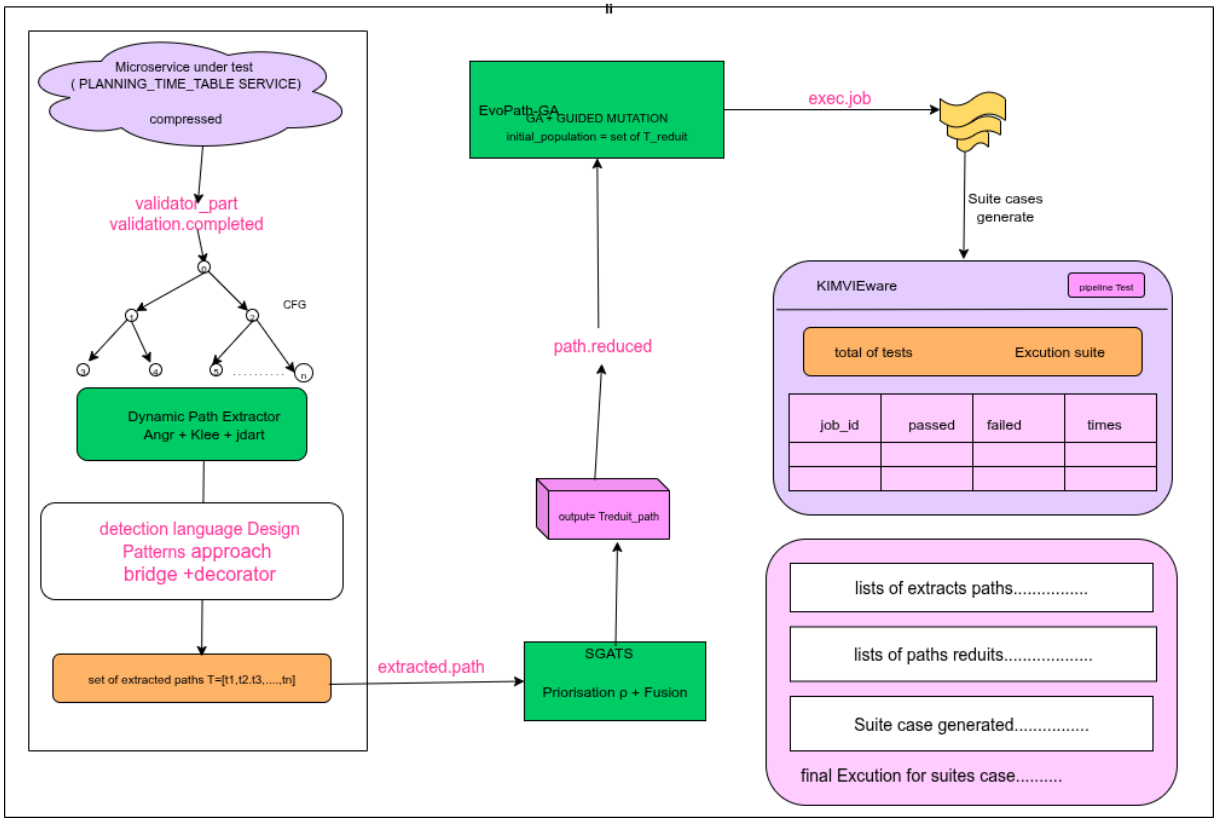

Figure 3.1 – Workflow for General Methodology approach.

Phase 0: Validator Service The initial **Input Gateway** receives and validates the compressed project. This microservice performs integrity checks and automatically detects the source language. Upon success, a unique `job_id` is assigned, and a message is published to the broker, effectively serving as the trigger to launch the resource-intensive Symbolic Execution of Phase 1.

Phase 1: Dynamic Path Extraction. The process begins by instrumenting the microservice under test and applying symbolic execution to explore its control flow graph (CFG) dynamically. The output is a set of execution paths, each annotated with corresponding path conditions.

Phase 2: SGATS Réduction. Given the exponential growth of paths, a **Selection Guide and Aggregation of Symbolic Trajectories (SGATS)** procedure reduces the search space. This réduction is guided by a priority function (to focus on critical paths) and a similarity function (to remove redundant or near-duplicate paths).

Phase 3: EvoPath-GA Optimization. The reduced path set is then optimized using an evolutionary algorithm. Candidate test cases are encoded as chromosomes, evolved through crossover and mutation, and evaluated against fitness functions that maximize

coverage and diversity while minimizing redundancy. Heuristic injection further enriches the population to avoid premature convergence.

Phase 4: Execution and Monitoring. The optimized test cases are executed within a microservices environment. Results, metrics (**including coverage, mutation score, and fault detection**), and execution traces are collected and presented on a real-time dashboard. Feedback loops connect this phase back to EvoPath-GA, allowing for the iterative improvement of test quality.

Integration. By combining symbolic execution, path réduction, genetic optimization, and microservices orchestration, KIMVIEware ensures scalability, modularity, and industrial applicability. The workflow not only addresses redundancy and path explosion but also explicitly integrates with CI/CD pipelines to bridge the academia–industry gap.

3.2 Phase 0: Project Reception and Validation (Input Gateway)

This initial phase acts as the **Input Gateway** for the KIMVIEware microservice architecture. Its primary role is to validate the integrity and compatibility of the submitted Subject Under Test (SUT) package before triggering the complex analysis workflow of Phase 1.

3.2.1 Validation of the Input Artifact

The input artifact, typically a compressed archive (e.g., ZIP or TAR), must satisfy several validation criteria:

1. **Compression Integrity Check:** The service verifies if the project archive is correctly compressed and fully accessible. An integrity failure immediately triggers a rejection message.
2. **Language Detection and Compatibility:** The system automatically analyzes the project structure (via file extensions and configuration files) to **determine the programming language** (e.g., Java, Python, C++) and checks this against the list of supported languages for the Symbolic Execution engine.
3. **Assignment of Job ID:** A unique identifier (`job_id`) is assigned to the request, ensuring complete traceability throughout the four phases of the microservice pipeline.

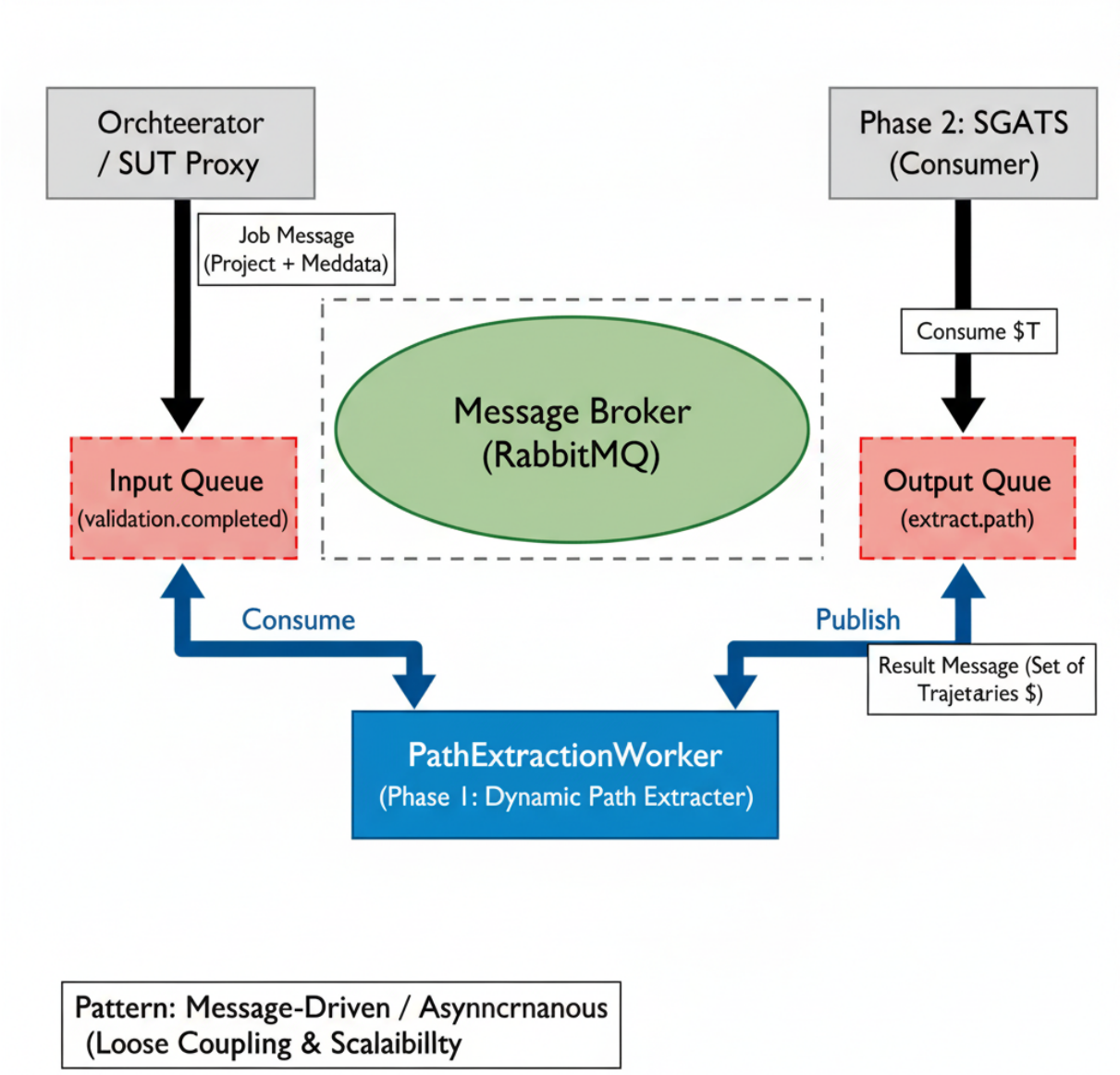

Figure 3.2 – Structural Design Patterns

3.2.2 Status Update and Triggering the Workflow

After successful validation, the service publishes the job status and essential metadata to the message broker, effectively launching Phase 1.

1. **Status Update:** The validation service updates the job status in the central database to `VALIDATED` or `READY_FOR_ANALYSIS`.
2. **Publishing the Launch Message:** A detailed message is serialized (containing the SUT location and the assigned `job_id`) and published to the message broker queue (e.g., `symbolic.job`), which serves as the trigger for the Symbolic Execution module (Phase 1).

Output Message Structure The message published to the broker for Phase 1 has the structure:

$$\text{Message} = \{\text{job\_id}, \text{SUT\_location}, \text{Detected\_Language}, \text{Status}\}$$

This message initiates the path generation process in Phase 1.

3.3 Phase 1: Dynamic Path Extraction

The first phase of the KIMVIEware pipeline is dedicated to building the **Control Flow Graph (CFG)** of each target microservice and extracting a representative, *feasible* set of execution trajectories T. This step is critical as it provides the search space for the subsequent réduction and optimization phases. The process combines static analysis (for CFG construction), dynamic instrumentation (for constraint collection), and **Symbolic Execution (SE)** integrated with an SMT solver (for feasibility checking and input generation).

3.3.1 Concepts and Definitions

To formalize the path extraction process, the following key concepts are employed:

1. **Control Flow Graph (CFG):** A directed graph $G = (V, E)$ where nodes $v \in V$ represent basic blocks of instructions, and edges $e \in E$ represent possible control transfers.
2. **Branch Set ($\mathcal{B}$):** The set of all conditional decision points (e.g., `if`, `switch` statements) extracted from the CFG. $|\mathcal{B}|$ denotes the total number of branches available for coverage.
3. **Trajectory (t):** An execution path through the CFG, from the entry node to an exit node.

4. **Path Condition ($\Pi(t)$):** The logical formula, accumulated through symbolic execution, representing the conjunction of constraints that must be satisfied by the input variables for execution to follow trajectory t.
5. **Feasible Trajectory (t):** A trajectory whose Path Condition $\Pi(t)$ is satisfiable, as determined by an SMT solver. The set T consists only of feasible trajectories.

3.3.2 Architecture and Communication Patterns

The **Dynamic Path Extractor** is implemented as an autonomous microservice, the `PathExtractionWorker`, which communicates with the orchestrator via a message broker (RabbitMQ). This pattern ensures resilience, scalability, and loose coupling.

Message-Driven Workflow

The workflow is explicitly message-driven, utilizing two dedicated message queues:

1. **Input Queue (`validation.completed`):** The orchestrator (or the SUT proxy) places a job message containing the compressed microservice project and metadata into this queue.
2. **Output Queue (`extract.path`):** Upon completion, the `PathExtractionWorker` serializes the resulting set of feasible trajectories T and publishes them to this queue, making them available for consumption by the next service in the pipeline (Phase 2: SGATS Réduction).

This design ensures that the entire extraction process is automatic and asynchronous, allowing the system to scale horizontally by deploying multiple `PathExtractionWorker` instances.

Structural Design Patterns

Two core design patterns govern the internal structure of the extractor for genericity: **Bridge** and **Decorator** (detailed in the previous draft and Figure 3.3).

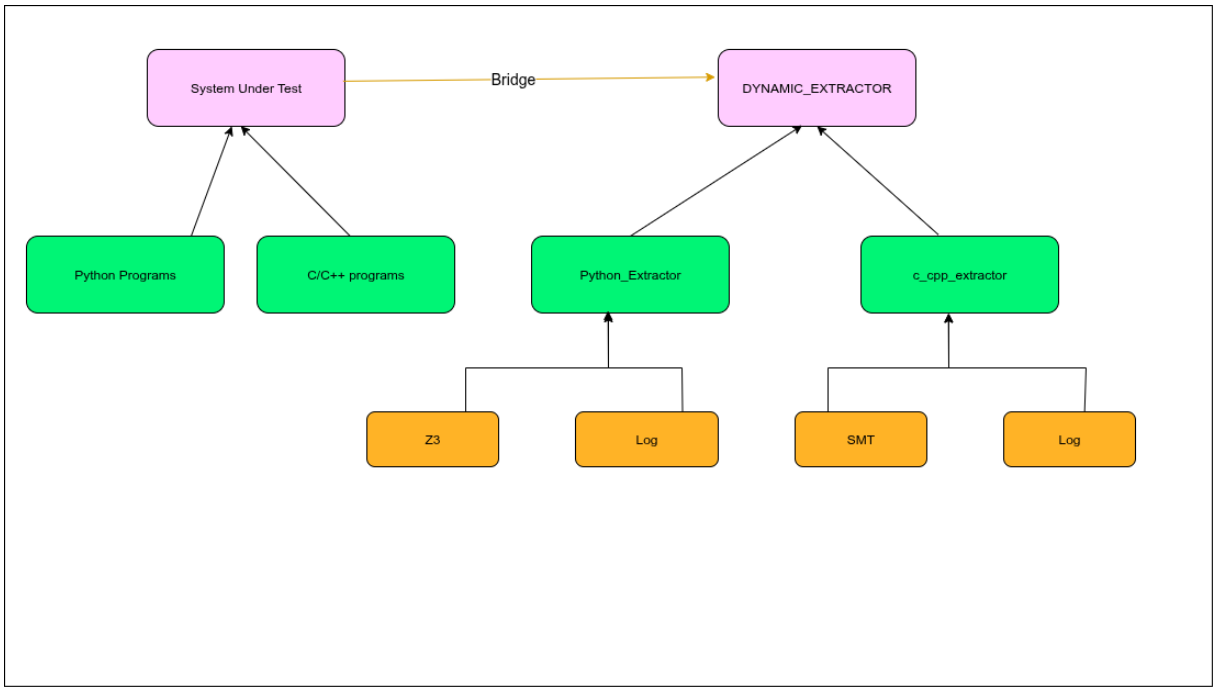

Figure 3.3 – Structural Design Patterns

3.3.3 Algorithm: Dynamic Path Extraction

The algorithmic process begins with consuming a job from the input queue, followed by the instantiation of the proper symbolic execution environment and the exploration loop.

Procedural Functions Detail. The specialized functions used for dynamic platform instantiation are defined as follows:

1. `DecompressAndAnalyze(`*payload*`):` Handles the extraction of the source code and configuration files from the input payload. It parses manifest files (e.g., `pom.xml`, `package.json`) to gather initial metadata required for the analysis.
2. `DetectLanguage(`*service_info*`):` Based on the analyzed metadata (file extensions, configuration structure), this function determines the primary programming language of the microservice (e.g., Java, Python, JavaScript).
3. `InstantiateExtractor(`*language*`):` This function implements the **Bridge Pattern** factory. It dynamically loads and returns the concrete extractor implementation corresponding to the detected `language` (e.g., if Java is detected, it returns the `JavaExtractor` instance).
4. `DecorateExtractor(`*base_extractor*`):` This function applies the **Decorator Pattern**, wrapping the concrete extractor with essential functionalities, such as the SMT solver integration module, logging, and instrumentation handlers, before the symbolic execution begins.

Algorithm 6: Dynamic Path Extractor Algorithm

Input: Input Queue Message M consumed from `service.sut`
Output: Message to Output Queue `extract.path` containing T

/* **Setup and Extractor Instantiation** */
`service_info` ← DecompressAndAnalyze$M.payload$
`language` ← DetectLanguage`service_info`
`base_extractor` ← InstantiateExtractor`language` /* *Bridge Pattern* */
`extractor` ← DecorateExtractor`base_extractor` /* *Decorator Pattern* */
`CFG` $G = (V, E)$ ← `extractor.buildCFG(service_info.codeSource)`
Initialize exploration queue $Q \leftarrow \{entry(G)\}$, feasible set $T \leftarrow \varnothing$
`StartTime` ← Current Time
/* **Symbolic Execution Loop** */
while $Q \neq \varnothing$ ***and*** $(Current\ Time - \texttt{StartTime}) < \Delta_{ext}$ **do**
  $t \leftarrow$ dequeue(Q) /* *May use a coverage-guided heuristic to prioritize* t */
  $\Pi(t) \leftarrow$ `extractor.generateConstraints(t)`
  if *SMT-Solver*$(\Pi(t))$ `=` ***SAT within*** τ_{SMT} **then**
    $T \leftarrow T \cup \{t\}$
    `extractor.enqueueSuccessors(t,` Q`)`
  else
    Discard t (infeasible or SMT timed out)
  end
end
/* **Output** */ `MessageOut` ← Serialize(T)
Publish `MessageOut` to `extract.path` queue
return `SUCCESS`

3.3.4 Theoretical Properties

The design and algorithmic choices in Phase 1 enforce strict properties regarding feasibility and termination, which are crucial for industrial applicability.

Theorem 3.1 (Feasibility Soundness)**.** *Every trajectory* $t \in T$ *returned by Algorithm 6 is guaranteed to be feasible, meaning its path condition* $\Pi(t)$ *is satisfiable.*

Proof. A path t is added to the set T only if the SMT solver returns `SAT` (satisfiable) for its path condition $\Pi(t)$ within the allotted time τ_{SMT}. By the definition of the SMT solver, a `SAT` result guarantees the existence of a concrete input assignment that satisfies $\Pi(t)$, thereby confirming the feasibility of the trajectory t. □

Theorem 3.2 (Budgeted Termination)**.** *Given a finite global timeout* Δ_{ext} *for the entire extraction process and a finite per-query timeout* τ_{SMT} *for the SMT solver, Algorithm 6 is guaranteed to terminate in finite time.*

Proof. Each iteration of the main `While` loop is bounded by two conditions: **1)** the global exploration budget Δ_{ext}, and **2)** the cost of the SMT query, which is bounded by τ_{SMT}.

Since the loop is controlled by Δ_{ext}, the total number of SMT calls and subsequent path explorations is capped. Thus, termination is guaranteed within a finite timeframe. □

3.4 Phase 2: SGATS Réduction

The initial set of feasible trajectories T generated by Symbolic Execution (Phase 1) is often excessively large and contains a significant degree of **semantic redundancy**. This redundancy—where multiple paths cover the same branches or exhibit highly similar execution structures—is a significant impediment to the scalability of the entire testing pipeline. Phase 2 introduces **SGATS** (*Selection-Guided Aggregation of Trajectories Symbolic*), a heuristic algorithm designed to drastically reduce the size of T by selecting and merging similar paths, while strictly preserving the overall code coverage.

3.4.1 Microservice Workflow and Communication

The SGATS process is encapsulated within the `PathReductionWorker` microservice. This worker operates autonomously, triggered by the arrival of the output from Phase 1, and facilitates loose coupling across the pipeline.

1. **Input Queue (`extract.path`):** The worker consumes a message containing the serialized set of initial trajectories T from this dedicated input queue.
2. **Output Queue (`reduce.path`):** Upon convergence, the worker serializes the compact, high-value set T_{reduced} and publishes it to this output queue, making it available for consumption by Phase 3 (EvoPath-GA Optimization).

3.4.2 Formal Notations and Metrics

Notation	Description
$\mathcal{B}$	**Set of all identifiable conditional branches** in the SUT's CFG.
$T = \{t_1, \ldots, t_N\}$	**Initial set of N feasible trajectories** (Phase 1).
$Br(t)$	**Set of branches visited** by trajectory t.
B_{seen}	**Set of branches already covered** by the current paths in T_{reduced}.
$\rho(t)$	**Priority score assigned** to trajectory t.
$sim(t_i, t_j)$	**Normalized similarity measure** between two paths (range 0..1).
τ	**Global similarity threshold** for fusion eligibility.
p	**Prefix similarity threshold** for fusion eligibility.
T_{reduced}	The final, **reduced set of high-value trajectories**.

Priority Function ($\rho(t)$)

The priority function $\rho(t)$ guides the selection process by ordering paths. It favors trajectories that offer maximal marginal coverage and structural novelty while penalizing those with high analysis cost.

$$\rho(t) = w_c \cdot \text{cov}(t) - w_\kappa \cdot \text{cost}(t) + w_u \cdot \text{Uniqueness}(t) \tag{3.1}$$

Où $w_c, w_\kappa, w_u \geq 0$ sont des poids ajustables, et les composantes sont :

1. **Marginal Coverage Gain ($\text{cov}(t)$):** $\frac{|Br(t) \setminus B_{seen}|}{|\mathcal{B}|}$
2. **Analysis and Execution Cost ($\text{cost}(t)$):** $c_0 + c_{SMT} \cdot |\Pi(t)| + c_{exec} \cdot |t|$
3. **Structural Uniqueness ($\text{Uniqueness}(t)$):** $1 - \max_{t' \in T_{reduced}} \left(sim(t, t') \right)$

Similarity Function ($\text{sim}(t_i, t_j)$)

The similarity function quantifies the structural and semantic resemblance between two trajectories t_i and t_j.

$$sim(t_i, t_j) = \alpha_p \cdot \frac{LCP(t_i, t_j)}{\min(|t_i|, |t_j|)} \; + \; \alpha_h \cdot \left(1 - \frac{Hamming(d_i, d_j)}{|d_i|} \right) \tag{3.2}$$

Where $\alpha_p + \alpha_h = 1$ are weighting parameters, and the components are formally defined below:

1. **LCP ($LCP(t_i, t_j)$): Longest Common Prefix** $LCP(t_i, t_j)$ is the length of the longest initial sequence of basic blocks (nodes) that trajectories t_i and t_j share before their paths diverge.

   $$LCP(t_i, t_j) = \max\{k \mid t_i[1..k] = t_j[1..k]\}$$

   The term $\frac{LCP(t_i,t_j)}{\min(|t_i|,|t_j|)}$ provides the normalized **structural similarity** of the common execution path.

2. **Hamming Distance ($Hamming(d_i, d_j)$):** Each path t is associated with a decision vector d, a sequence of Boolean outcomes for conditional branches. d_i and d_j are the decision vectors of t_i and t_j, respectively, truncated to the length of the common prefix $LCP(t_i, t_j)$. The Hamming distance counts the number of positions where the decisions differ.

   $$Hamming(d_i, d_j) = \sum_{k=1}^{LCP(t_i,t_j)} \mathbb{I}(d_i[k] \neq d_j[k])$$

   The value $1 - \frac{Hamming(d_i,d_j)}{|d_i|}$ measures the **semantic coherence**: low Hamming

distance indicates that the paths follow the same constraints up to the point of divergence.

Fusion Criterion (fuseable)

A trajectory t is considered **fusion-eligible** with t' only if both the global similarity and the prefix similarity exceed defined thresholds (τ and p).

$$\text{fuseable}(t_i, t_j, \tau, p) \iff (sim(t_i, t_j) \geq \tau) \wedge \left(\frac{LCP(t_i, t_j)}{\min(|t_i|, |t_j|)} \geq p \right)$$

State-Based Fusion Detection: Fusion Hit Criterion

Beyond structural similarity (measured by $\text{sim}(t_i, t_j)$), SGATS employs a **state-based redundancy detection mechanism** called the **Fusion Hit** criterion. This mechanism identifies trajectories that reach semantically equivalent program states, even if their syntactic paths differ.

Definition 3.1 (Program State at Decision Point)**.** *Let $n \in V$ be a decision node (conditional branch) in the CFG $G = (V, E)$. The **symbolic state** at n for trajectory t is defined as:*

$$\Sigma(t, n) = \langle \sigma_t(n), \Pi_t(n) \rangle$$

where:

1. *$\sigma_t(n)$ is the symbolic store (mapping of program variables to symbolic expressions) when t reaches node n.*
2. *$\Pi_t(n)$ is the accumulated path condition up to node n.*

Definition 3.2 (Fusion Hit)**.** *A **Fusion Hit** occurs when two distinct trajectories t_i and t_j arrive at the same program point n with **semantically equivalent states**:*

$$\text{FusionHit}(t_i, t_j, n) \iff (t_i \neq t_j) \wedge \text{StateEquiv}(\Sigma(t_i, n), \Sigma(t_j, n))$$

where StateEquiv *is a semantic equivalence predicate checked via SMT solver queries:*

$$\text{StateEquiv}(\Sigma_1, \Sigma_2) \iff \textit{SMT-Solver}\left(\Pi_1 \wedge \Pi_2 \wedge \bigwedge_{v \in Vars} (\sigma_1(v) = \sigma_2(v)) \right) = \textit{SAT}$$

Operational Semantics and Early Termination. When a Fusion Hit is detected during symbolic exploration (Phase 1) or reduction (Phase 2), the system applies the following rule:

1. **Test Case Generation:** A concrete test input is immediately synthesized from the satisfying assignment of $\Pi_t(n)$ using the SMT solver.

2. **Path Truncation:** The trajectory t is **truncated** at node n, as further exploration would be redundant with the previously explored path that reached the same state.
3. **Coverage Credit:** The branches $Br(t)$ covered up to node n are credited to the test suite, but successors of n are **not re-explored**.

This mechanism provides **dynamic redundancy elimination** at the state level, complementing the structural redundancy elimination of the fuse operator.

Integration with SGATS Priority Function. Paths that trigger Fusion Hits are assigned a **penalty** in the priority function $\rho(t)$ (Equation 3.1), as they contribute less marginal coverage:

$$\rho(t) = w_c \cdot \text{cov}(t) - w_\kappa \cdot \text{cost}(t) + w_u \cdot \text{Uniqueness}(t) - \mathbf{w_f} \cdot \text{FusionPenalty}(\mathbf{t})$$

where:

$$\text{FusionPenalty}(t) = \frac{\text{Number of Fusion Hits encountered by } t}{\text{Total decision points visited by } t}$$

and $w_f \geq 0$ is a tunable weight (typically $w_f = 0.1$).

Algorithmic Impact. The Fusion Hit detection is implemented as a **memoization table** $\mathcal{H} : V \to \{\Sigma\}$ that stores previously encountered states at each decision node. During path exploration (Algorithm 6) or reduction (Algorithm 7), each new state $\Sigma(t, n)$ is checked against $\mathcal{H}(n)$ before enqueuing successors.

The Fusion Hit criterion is particularly effective in programs with **loops** and **deep recursion**, where multiple paths can converge to identical states after different iteration counts. It acts as a **dynamic subsumption** check, preventing the exploration of provably redundant execution branches.

3.4.3 SGATS Algorithm: Selection-Guided Aggregation

The SGATS algorithm implements the selection and aggregation process.

Algorithm 7: SGATS

Input: Input Message M from `extract.path`, thresholds τ, p

Output: Message to `reduce.path` containing $T_{reduced}$

/* **Initialization and Sorting** */ $T \leftarrow$ DeserializeMessage(M)
$F \leftarrow$ priority_queue(T, key $= \rho$)
$T_{\text{reduced}} \leftarrow \emptyset$
$B_{seen} \leftarrow \emptyset$
$\mathcal{H} \leftarrow \emptyset$ /* ***State memoization table for Fusion Hit detection*** */
/* **Selection and Fusion Loop** */ **while** $F \neq \varnothing$ **do**
  $t \leftarrow$ extract_max(F)
  /* **NEW: Fusion Hit Check** */ **foreach** *decision node* n *in* t **do**
    if FusionHit*(t, $\mathcal{H}(n)$, n)* **then**
      Truncate t at node n
      Generate test input from $\Pi_t(n)$
      break
    end
    $\mathcal{H}(n) \leftarrow \mathcal{H}(n) \cup \{\Sigma(t, n)\}$
  end
  `BestTarget` $\leftarrow$ FindBestFusionTarget($t, T_{\text{reduced}}, \tau, p$)
  if *`BestTarget` exists* **then**
    /* ... fusion logic ... */
  else
    Insert t into T_{reduced}
    $B_{seen} \leftarrow B_{seen} \cup Br(t)$
  end
end
`MessageOut` $\leftarrow$ Serialize($T_{reduced}$)
Publish `MessageOut` to `reduce.path` queue
return `SUCCESS`

3.4.4 Theoretical Properties

Theorem 3.3 (Coverage Preservation)**.** *Let $B_T = \bigcup_{t \in T} Br(t)$ be the branch coverage of the initial set, and $B_R = \bigcup_{t' \in T_{reduced}} Br(t')$ the coverage of the reduced set. Then:*

$$B_T \subseteq B_R.$$

Proof. The proof is constructive. The fuse operator is explicitly designed to ensure that the merged path t_{new} contains the union of branches from the original paths: $Br(t_{new}) \supseteq$

$Br(t) \cup Br(t')$. Since every path in T is either directly selected or merged into a coverage-preserving representative in $T_{reduced}$, no branch covered by the initial set is lost. □

Theorem 3.4 (Polynomial Complexity)**.** *Let $N = |T|$ be the initial number of trajectories and L the average path length. The computational complexity of SGATS is bounded by $O(N^2 \cdot L)$.*

Proof. The search for the optimal fusion target within the main loop primarily drives the complexity. The algorithm iterates N times. In iteration i, $|T_{\text{reduced}}|$ is $O(i)$. Finding the best fusion target requires $O(i)$ comparisons, where each comparison (calculating *sim* and fuseable) takes $O(L)$ time. The total complexity sums up as $\sum_{i=1}^{N} O(i \cdot L) = O(L) \cdot O(N^2)$. □

3.4.5 Remark

SGATS effectively manages the path redundancy problem by combining selection heuristics (priority ρ) and aggregation logic (similarity *sim*). It provides an **efficient** (polynomial complexity) and **safe** (coverage preservation) réduction of the path space, generating the highly-optimized input set T_{reduced} for Phase 3.

3.5 Phase 3: EvoPath-GA Optimization (Adaptive Meta-heuristic)

The objective of Phase 3 is to solve the **Test Suite Minimization Problem** while ensuring coverage and quality. This challenge is formalized as a **Multi-Objective Optimization Problem (MOOP)**. The input, the reduced set of paths T_{reduced} from Phase 2, is explored by the **EvoPath-GA** (*Evolutionary Path Genetic Algorithm*), which balances four contradictory objectives: **maximization of coverage**, **maximization of quality (Mutation Score)**, and **minimization of size and execution cost**.

3.5.1 Meta-heuristic Nature of EvoPath-GA

EvoPath-GA is not a standard Genetic Algorithm (GA) but a **Hybrid Meta-heuristic** specifically engineered for the context of reduced symbolic test sets. Its enhancements over a classical GA provide faster convergence towards an operational, rather than theoretical, optimum.

1. **Exploitation of the Reduced Search Space (T_{reduced}):** A classical GA must explore the exponential combinatorial space of all raw paths T. EvoPath-GA, thanks to the work of Phase 2 (SGATS), operates solely on T_{reduced}. This

drastically reduces complexity and ensures the search space is already **feasible, non-redundant, and prioritized**.

2. **Adaptive Multi-Objective Fitness Function (MOOP):** Classical GA typically optimizes a single objective (e.g., coverage). EvoPath-GA integrates four contradictory objectives, including **cost** and **Mutation Score**, which are essential in an industrial environment. Furthermore, the weights of this function are dynamically adjusted by the Feedback Loop of Phase 4.

3. **Heuristic Seeding Strategy:** Instead of random initialization, EvoPath-GA injects high-quality heuristic chromosomes into its initial population P_0, such as **Greedy Coverage Seeds** (minimal sets for coverage) and **Priority Seeds** (sets based on the SGATS priority $\rho(t)$). This reduces the number of generations needed to achieve high fitness.

4. **Dynamic Anti-Cloning Mechanism ($\delta(g)$):** To combat **premature convergence** (stagnation), EvoPath-GA uses a dynamic similarity threshold $\delta(g)$ to merge or eliminate overly similar individuals at each generation g.

$$\delta(g) = \delta_{\min} + (\delta_{\max} - \delta_{\min}) \cdot (1 - \text{progress}(g))$$

Explanation: When the progress progress(g) of the best solution is low (stagnation), the threshold is high ($\delta_{\max}$), favoring clone elimination and promoting **exploration**. When progress is strong, the threshold is low ($\delta_{\min}$), allowing the **exploitation** of small beneficial variations.

3.5.2 Chromosome Encoding (Subset Selection Model)

As the problem is a subset selection from T_{reduced}, the encoding must be binary.

Definition 3.3 (Chromosome Structure)**.** *Let $N' = |T_{reduced}|$ be the size of the reduced path set. A chromosome C representing a candidate test suite is encoded as a binary vector of length N':*

$$C = (c_1, c_2, \ldots, c_{N'}), \quad c_i \in \{0, 1\}$$

*where gene $c_i = 1$ if path $t_i \in T_{reduced}$ is **included** in the candidate test suite S, and $c_i = 0$ otherwise.*

Justification: This **Binary Selection** encoding is the simplest and most efficient for standard genetic operators in Set Covering type problems.

3.5.3 Multi-Objective Fitness Function $\mathcal{F}(C)$

The fitness function $\mathcal{F}(C)$ is the core of EvoPath-GA. It is a weighted linear combination of objectives, aiming for maximization.

Definition 3.4 (Scalarized Fitness Function $\mathcal{F}(C)$)**.**

$$\mathcal{F}(C) = w_{cov} \cdot f_{cov}(C) + w_{div} \cdot f_{div}(C) + w_{\mu} \cdot f_{\mu}(C) - w_{cost} \cdot f_{cost}(C) \tag{3.3}$$

where $w_{cov}, w_{div}, w_{\mu}, w_{cost} \geq 0$ *are the importance weights (with* $\sum w_i = 1$*).*

The four objective components are formally defined and explained:

1. **Coverage Objective (f_{cov}): Maximization** It measures the proportion of total branch coverage achieved by the suite S represented by C.

$$f_{\text{cov}}(C) = \frac{1}{|\mathcal{B}|} \cdot \left| \bigcup_{i:c_i=1} Br(t_i) \right|$$

   Explanation: $|\mathcal{B}|$ is the total number of branches in the SUT. $\left|\bigcup_{i:c_i=1} Br(t_i)\right|$ is the number of unique branches covered by all selected paths ($c_i = 1$). Normalization ensures $f_{\text{cov}} \in [0, 1]$.

2. **Diversity Objective (f_{div}): Maximization** It evaluates the uniqueness of chromosome C relative to the population P, encouraging exploration and solution variety.

$$f_{\text{div}}(C) = 1 - \frac{1}{|P|} \sum_{C' \in P} \text{sim}_{\text{set}}(C, C')$$

   Explanation: It is calculated as the inverse of the average similarity between C and all other chromosomes C' in population $|P|$. $\text{sim}_{\text{set}}(C, C')$ uses a metric like the Jaccard index on the chosen path subsets. A score closer to 1 means C is more unique.

3. **Quality/Mutation Objective (f_{μ}): Maximization** This is the measure of industrial quality (defect detection capability). It is based on the **Mutation Score** (μ) measured in Phase 4.

$$f_{\mu}(C) = \text{HistoricalMutationScore}(C) = \frac{\text{Killed Mutants}}{\text{Non-Equivalent Mutants}}$$

   Explanation: $f_{\mu}(C)$ is the average historical mutation score of the paths in C. A high score indicates that the test suite is **robust** and effective at killing mutants (simulated defects). The inclusion of this term ensures that optimization targets not just coverage, but **defect detection quality**.

4. **Cost Objective (f_{cost}): Minimization (Penalty)** It penalizes test suites that are costly in resources, guiding the solution toward economic efficiency.

$$f_{\text{cost}}(C) = \frac{\sum_{i:c_i=1} \text{cost}(t_i)}{\max_{C' \in P} \sum_{j:c'_j=1} \text{cost}(t_j)}$$

Explanation: The numerator is the sum of the estimated costs (or Phase 4 adjusted costs) of the selected paths. Normalization by the maximum cost in the population ensures $f_{\text{cost}} \in [0, 1]$. This term is subtracted from $\mathcal{F}(C)$, creating **selective pressure** for size and cost minimization.

3.5.4 EvoPath-GA Theorems and Proofs

Theorem 3.5 (Elitist Monotonicity)**.** *If an elitism strategy is employed, preserving the best e individuals at each generation, then the best fitness score in the population is non-decreasing across generations:*

$$\max_{C \in P_{g+1}} \mathcal{F}(C) \geq \max_{C \in P_g} \mathcal{F}(C) \quad \forall g.$$

Proof. The elitism step (Elitism) ensures that the e chromosomes with the $\max \mathcal{F}(C)$ in P_g are copied directly into P_{g+1}. Even if the new descendants P_{new} generated by Crossover and Mutation are all of lower quality, the $\max \mathcal{F}(C)$ from P_g is preserved, ensuring non-decreasing performance. □

Theorem 3.6 (Asymptotic Optimality (Convergence))**.** *Given a finite search space (defined by $T_{reduced}$), a strictly positive mutation probability $p_m > 0$, and the use of elitism, EvoPath-GA converges to the global optimum C^* with a probability of 1 as the number of generations G tends towards infinity:*

$$\lim_{G \to \infty} \Pr\left(\exists C \in P_G : \mathcal{F}(C) = \mathcal{F}(C^*)\right) = 1.$$

Proof Sketch (Markov Chain). The genetic algorithm can be modeled as a homogeneous Markov chain with a finite state space. The condition $p_m > 0$ (mutation probability) ensures that it is possible to transition from any state (chromosome) to any other state, including the global optimum C^*, in a finite number of steps. The transition matrix is therefore irreducible. Furthermore, the elitism property guarantees that the state C^* is an absorbing or persistent state. Consequently, the probability of reaching the optimal state (convergence) tends to 1. □

3.5.5 EvoPath-GA Optimization Algorithm (Algorithm)

Algorithm 8: EvoPath-GA Optimization Algorithm (Hybrid Meta-heuristic)

Input: Reduced set T_{reduced}, population size P_{size}, max generations G_{max}, weights $w_{\text{cov}}, w_{\text{div}}, w_{\text{cost}}, w_{\mu}$

Output: Optimal Test Suite S

/* **Initialization (Phase 3 Startup)** */
$T_{\text{reduced}} \leftarrow$ DeserializeMessage(Input Message);
$P_{current} \leftarrow$ InitializePopulation(P_{size});
$P_{current} \leftarrow$ InjectHeuristicSeeds($P_{current}$) /* *Injection of high-priority solutions (Heuristic Seeding)* */;
$g \leftarrow 0$;
/* **Evolutionary Loop** */
while $g < G_{max}$ ***and*** *termination criterion not met* **do**
    $P_{new} \leftarrow \emptyset$;
    foreach $C \in P_{current}$ **do**
        Evaluate fitness $\mathcal{F}(C)$ /* *Using the multi-objective $\mathcal{F}(C)$ (Eq. 3.3)* */;
    end
    for $i \leftarrow 1$ **to** $P_{size}/2$ **do**
        $P_1 \leftarrow$ TournamentSelection($P_{current}$);
        $P_2 \leftarrow$ TournamentSelection($P_{current}$);
        $(O_1, O_2) \leftarrow$ TwoPointCrossover(P_1, P_2);
        $O_1 \leftarrow$ Mutate(O_1) /* *Bit-Flip Mutation, $p_m > 0$* */;
        $O_2 \leftarrow$ Mutate(O_2);
        $P_{new} \leftarrow P_{new} \cup \{O_1, O_2\}$;
    end
    $P_{current} \leftarrow$ Elitism($P_{current}$) /* *Preservation of the best e individuals (Monotonicity guaranteed)* */;
    $P_{merged} \leftarrow P_{current} \cup P_{new}$;
    $P_{g+1} \leftarrow$ AntiCloneSelection($P_{merged}, \delta(g)$) /* *Clone elimination (Diversity guaranteed)* */;
    $g \leftarrow g + 1$;
end
/* **Microservice Output** */ $S \leftarrow$ chromosome with max $\mathcal{F}(C)$ in $P_{G_{max}}$;
`MessageOut` $\leftarrow$ Serialize(S) /* *Optimized set of path constraints* */;
Publish `MessageOut` to `exec.job` queue;
return `SUCCESS`;

3.6 Phase 4: Execution, Data Collection, and Analysis (Mutation-Guided)

This phase finalizes the optimized test suite S by executing it concretely. The key concepts here are the **Operational Feedback Loop** and the quality assessment via **Mutation Testing**.

3.6.1 Objectives and Role of Mutation Testing

Phase 4 transcends simple execution to become a test bed for the **quality** and **economic efficiency** of S. The integration of **Mutation Testing** (μ) directly addresses the need to validate the test suite's defect detection capability, a fundamental criterion for the industry.

Definition 3.5 (Mutation Testing)**.** *Mutation Testing is a software quality assurance method that evaluates the* ***sufficiency*** *of a test suite by introducing small defects (mutants) into the source code (e.g., changing $>$ to $\geq$). If the test suite fails against the mutant code (it "kills" the mutant), it proves its ability to detect subtle faults.*

3.6.2 Artifact Collection and Traceability (Including Mutation Score)

Each execution generates an **Execution Artifact** $\mathcal{A}$, ensuring complete traceability of the environment, performance, and quality.

Definition 3.6 (Execution Artifact Set ($\mathcal{A}$))**.** *For every executed test s_i, the artifact $\mathcal{A}(s_i)$ stored in the NoSQL database (e.g., MongoDB) contains:*

$$\mathcal{A}(s_i) = \{\mathbb{I}(s_i), \textbf{\textit{Coverage}}(s_i), \textbf{\textit{Time}}(s_i), \textbf{\textit{Cost}}(s_i), \textbf{MutationScore}(\mathcal{M}), SUT\_Version, Logs\}$$

The key metrics extracted are:

1. **Actual Execution Cost (κ_{act}):** Empirical measurement of CPU time, memory, and execution duration. This is used to **calibrate** the estimated cost $\text{cost}(t)$ from Phase 3.
2. **Mutation Score (μ):** The percentage of non-equivalent mutants killed by the test suite S during its execution. This is the measured value of the objective $f_\mu(C)$ under real conditions.

3.6.3 The Operational Fitness Tracker: Bridging the Academia-Industry Gap

The **Operational Fitness Tracker** is the supervision dashboard of KIMVIEware. It serves as the explicit interface between the results of academic optimization (EvoPath-GA) and industrial performance and quality requirements (CI/CD). It translates optimization concepts into actionable **Key Performance Indicators (KPIs)**.

Transformation of Objectives into Industrial KPIs.

1. **Quality (Mutation Score μ):** The score μ is displayed in real-time. It acts as a **Quality Gate**: if μ falls below a defined threshold (e.g., 90%), code merging is blocked, justifying the effectiveness of the multi-objective $\mathcal{F}(C)$.
2. **Economic Efficiency (Cost Reduction Ratio):** This quantifies the advantage of the minimization in Phase 3. This KPI measures resource savings compared to executing the raw set T:

$$\text{Ratio} = \frac{\sum Cost(T) - \sum Cost(S)}{\sum Cost(T)}$$

3. **Throughput (Time-to-Coverage - TTC):** Measures the time required to reach a target coverage level ($X\%$). This is an essential performance KPI for Continuous Integration (CI) pipelines.

3.6.4 Operational Feedback Loop

The feedback loop is the mechanism by which the empirical results from Phase 4 are injected into Phase 3, transforming EvoPath-GA into an **self-adaptive** system.

1. **Mechanism:** The `ExecutionWorker` publishes the observed metrics (κ_{act}, μ) to the `ga.feedback` queue. The `PathOptimizerWorker` (Phase 3) periodically consumes these messages.
2. **Adaptive Refinement of Fitness Weights:** Real data is used to dynamically adjust the weights of the $\mathcal{F}(C)$ equation:
   (a) **Cost Weight Adjustment (w_{cost}):** If the gap between predicted cost and actual cost (κ_{act}) is too large, w_{cost} is increased to more severely penalize costly paths in the future.
   (b) **Mutation Guidance (w_μ):** If the Mutation Score μ is low, the weight w_μ is temporarily increased. This forces the next generation of EvoPath-GA to select

paths that have historically been more effective at killing mutants, **guiding the optimization towards better defect detection quality**.

This loop ensures that the EvoPath-GA model optimizes not just for theoretical estimates, but for actual operational performance.

3.7 Complete Pipeline Synthesis

The effectiveness of the KIMVIEware platform stems from the strategic orchestration of its four distinct phases into a unified, asynchronous processing pipeline. This final section synthesizes the complete methodology, demonstrating how Symbolic Execution (Phase 1) is coupled with Réduction Heuristics (Phase 2), Evolutionary Optimization (Phase 3), and Scalable Execution (Phase 4).

3.7.1 KIMVIEware Global Pipeline Algorithm

Algorithm 9 provides the high-level representation of the end-to-end process, showcasing the data transformation across the architectural phases.

Algorithm 9: KIMVIEware Global Pipeline

Input: System Under Test (SUT) Configuration

Output: Optimized, runnable Test Suite S

/* **Phase 1: Dynamic Path Extraction** */
$T \leftarrow$ DynamicPathExtractor(SUT) /* *Initial, redundant set of feasible paths.* */;

/* **Phase 2: SGATS Reduction** */ $T_{\text{reduced}} \leftarrow \text{SGATS}(T)$ /* *Compact set with preserved coverage,* $T_{reduced} \subseteq T$. */;

/* **Phase 3: EvoPath-GA Optimization** */ $S \leftarrow \text{EvoPath\_GA}(T_{\text{reduced}})$ /* *Optimal subset of paths maximizing Fitness* $\mathcal{F}(C)$. */;

/* **Phase 4: Execution, Data Collection & Feedback** */
ExecuteAndAnalyze(S) /* *Generates concrete inputs, runs tests, and feeds back performance data.* */;

return S /* *The final set of path constraints, ready for deployment.* */;

3.7.2 Phase-by-Phase Integration and Justification

The four phases are designed to address the sequential challenges inherent in automated test generation for complex systems:

1. **Phase 1 (Dynamic Path Extraction):** The process begins with Symbolic Execution (SE), which systematically explores the SUT's Control Flow Graph (CFG)

to generate T, a set of path constraints and corresponding execution trajectories. This phase addresses the fundamental challenge of generating high-coverage and feasible inputs.

2. **Phase 2 (SGATS Reduction):** T suffers from **path explosion** and high redundancy. The SGATS heuristic addresses this by employing a priority-guided selection (ρ) and a similarity-based aggregation (sim) to produce the compact set T_{reduced}. This step is essential for converting the non-scalable output of SE into a tractable search space for the subsequent GA.
3. **Phase 3 (EvoPath-GA Optimization):** The problem is now reduced to Test Suite Minimization. EvoPath-GA, a multi-objective algorithm, searches T_{reduced} to find the optimal subset S. This phase balances coverage maximization, cost minimization, and diversity maintenance to yield the final, economically efficient test suite S.
4. **Phase 4 (Execution and Feedback):** The optimized suite S is materialized into concrete inputs and executed in a scalable, microservice environment. This phase validates the feasibility of S and, critically, closes the optimization loop by feeding real-world cost and coverage data back to Phase 3, enabling adaptive optimization in subsequent runs.

3.7.3 Theoretical Guarantees of the Pipeline

Theorems concerning the pipeline's behavior can be established based on the properties of its constituent phases.

Theorem 3.7 (Pipeline Termination)**.** *If each phase respects finite resource budgets (e.g., SMT budget in Phase 1, maximum generations $G_{\max}$ in Phase 3, and time/SMT budgets in Phase 4), then Algorithm 9 terminates in finite time.*

Proof. Termination is guaranteed by bounding the complexity of each phase:

1. **a finite budget on SMT solver terminates Phase 1 (Extraction):** The symbolic execution process calls or path depth.
2. **Phase 2 (SGATS):** Terminating complexity is $O(|T|^2 \cdot L)$, which is polynomial and thus finite.
3. **Phase 3 (EvoPath-GA):** Termination is guaranteed by setting a maximum number of generations $G_{\max}$.
4. **Phase 4 (Execution):** Each test run has a defined time and resource budget, guaranteeing a finite duration for the set S.

Since all sub-components terminate in finite time, the overall pipeline terminates in finite time. □

Theorem 3.8 (Coverage Monotonicity and Bound)**.** *Let B_T be the coverage of the initial set T (Phase 1), $B_{reduced}$ be the coverage of $T_{reduced}$ (Phase 2), and B_S be the coverage of the final suite S (Phase 3). The pipeline guarantees:*

$$B_S \subseteq B_{reduced} = B_T.$$

In practice, S is designed to approximate the full coverage of $B_{reduced}$ by maximizing $f_{cov}(C)$.

Proof.

1. The equality $B_{\text{reduced}} = B_T$ is proven by the Coverage Preservation Theorem of the SGATS algorithm (Phase 2), which guarantees that no coverage achieved by T is lost during the réduction to T_{reduced}.
2. The containment $B_S \subseteq B_{\text{reduced}}$ holds because the EvoPath-GA (Phase 3) is a subset selection algorithm, choosing $S \subseteq T_{\text{reduced}}$. Therefore, the branches covered by S must be a subset of those covered by T_{reduced}.

The entire pipeline is therefore coverage-safe up to the output of Phase 2. The objective of Phase 3 is to minimize the size of the set while holding B_S as close as possible to B_{reduced}. □

3.8 KIMVIEware Platform Architecture

The KIMVIEware platform is realized as a distributed, **event-driven microservice architecture**. This architectural choice is foundational, as it directly addresses the scalability and resource isolation requirements necessary for handling the high computational loads of symbolic execution and large-scale test execution in microservices environments.

3.8.1 Architectural Paradigm: Microservices

The platform adheres to the microservice paradigm, ensuring that each core function (Extraction, Reduction, Optimization, Execution) operates as an independent, loosely coupled service.

1. **Decoupling:** Each phase is encapsulated in its own worker service, communicating exclusively through a central message broker. This prevents cascading failures and simplifies development and maintenance.
2. **Scalability:** Individual services can be scaled horizontally (e.g., adding more `ExecutionWorker` instances) based on specific workload demands without affecting the other components.
3. **Technology Heterogeneity:** The decoupled nature allows specialized components (like the SMT solvers in Phase 1 or the GA engine in Phase 3) to run in their optimal environments.

3.8.2 Component Breakdown

The architecture is composed of three main layers: the **Processing Layer (Workers)**, the **Communication Layer (Message Broker)**, and the **Data/Storage Layer**.

Processing Layer (Workers)

The four phases of the methodology are implemented as dedicated worker microservices:

1. **`PathExtractorWorker` (Phase 1):** Consumes the initial SUT configuration. Executes the Symbolic Execution (SE) engine. Produces the raw set of feasible paths T.
2. **`PathReductionWorker` (Phase 2):** Consumes the high-volume path set T. Runs the SGATS algorithm to produce the compact set T_{reduced}.
3. **`PathOptimizerWorker` (Phase 3):** Consumes T_{reduced}. Executes the EvoPath-GA algorithm to find the optimal test suite subset S.
4. **`ExecutionWorker` (Phase 4):** Consumes the optimized suite S. Runs the tests in isolated, containerized environments. Collects all execution artifacts and metrics.

Communication Layer (Message Broker)

A central Message Broker (e.g., Kafka or RabbitMQ) manages the asynchronous flow of data, decoupling producers (the output of a phase) from consumers (the input of the next phase). Three primary queues define the pipeline flow:

1. **`extract.path` Queue:** Output of Phase 1 (T). Consumed by the `PathReductionWorker`.
2. **`reduce.path` Queue:** Output of Phase 2 (T_{reduced}). Consumed by the `PathOptimizerWorker`.
3. **`exec.job` Queue:** Output of Phase 3 (The optimized suite S). Consumed by the `ExecutionWorker` pool.

Data and Storage Layer

The data layer is partitioned based on the type and volume of data:

1. **Artifact Database (MongoDB):** Stores structured, non-relational data, primarily the Execution Artifacts (Section 3.6) and metadata for high-speed query and retrieval.
2. **Object Storage (MinIO):** Dedicated store for large binary artifacts, such as execution logs, trace files, and container crash dumps, referenced via URI in the Artifact Database.
3. **Metrics Database (Prometheus):** Collects time-series data (CPU usage, queue latency, test run times) for real-time monitoring and performance analysis.

3.8.3 End-to-End Execution Flow

Figure 3.4 illustrates the sequential data flow and the asynchronous communication pattern. The platform operates as a continuous, closed-loop system:

1. The process is initiated by an external trigger (CI/CD webhook) submitting the SUT to the `PathExtractorWorker`.
2. Data is progressively transformed and reduced as it passes through the queues from `extract.path` to `reduce.path` to `exec.job`.
3. The `ExecutionWorker` pool consumes the final optimized jobs and generates the Execution Artifacts.
4. The **Feedback Loop** (indicated by the dashed line) ensures that the analyzed results (e.g., actual execution cost and new coverage) are sent back to the `PathOptimizerWorker` to refine the fitness function parameters for the next iteration.

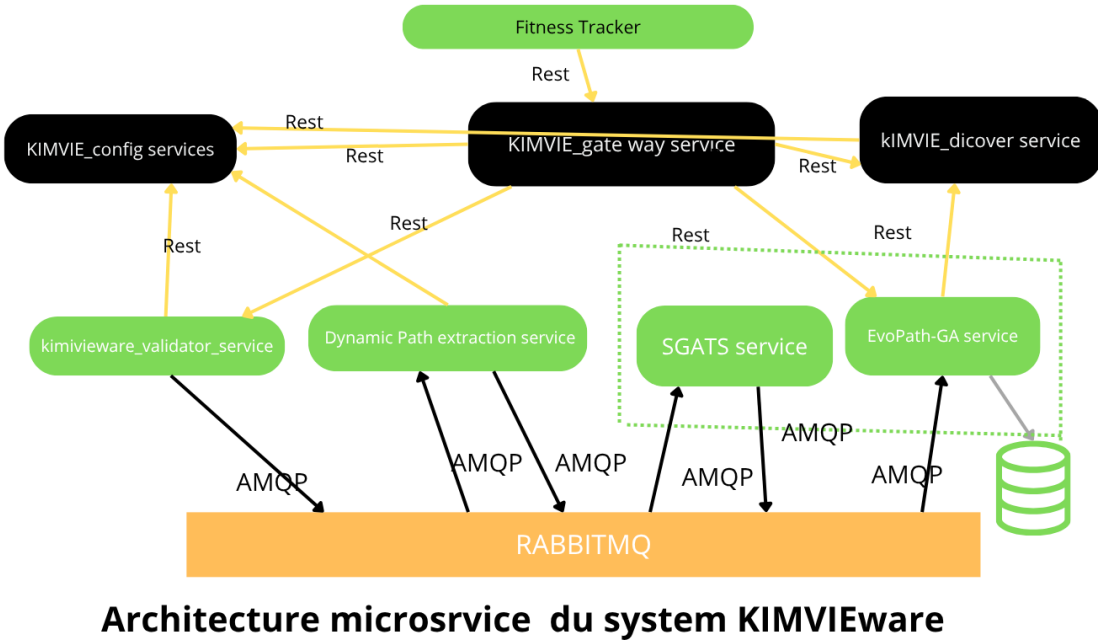

Figure 3.4 – KIMVIEware Platform Microservice Architecture. The flow is unidirectional from Phase 0 to Phase 4, with a critical feedback loop connecting execution analysis back to the EvoPath-GA optimization engine.

The microservice architecture thus guarantees high throughput and fault isolation, making KIMVIEware suitable for industrial-scale deployment, where scalability and resource management are paramount concerns.

Chapter 4

Results and Experimental Analysis

4.1 Empirical Evaluation of KIMVIEware

This section presents the empirical evaluation of the **KIMVIEware** platform across its four execution phases: Symbolic Execution (Phase 1), Trace Simplification (Phase 2), Optimization (Phase 3), and Execution (Phase 4). All reported results represent the mean of 5 independent runs with 95% confidence intervals, ensuring reliability.

4.1.1 Path Reduction Results (SGATS - Hypothesis H1)

Hypothesis H1 stated that the Symbolic-Guided Abstract Trace Simplification (**SGATS**) component would achieve a $\geq$ 80% reduction in the number of generated paths while maintaining 100% coverage. The results are presented in Table 4.1.

Table 4.1 – SGATS Path Reduction Efficiency (Phase 2)

Benchmark	$\lvert T\rvert$ **(Initial)**	$\lvert T_{\mathbf{red}}\rvert$ **(SGATS)**	**Reduction Rate**	**Coverage Preserved**	**Time (s)**
C/C++ Programs (KLEE)					
copy.c	847	124	85.4%	100%	23.7
echo.c	1,156	189	83.7%	100%	41.2
cut.c	1,523	267	82.5%	100%	67.8
printf.c	2,341	198	91.5%	100%	112.4
Python Programs (AST/Angr)					
triangle.py	156	28	82.1%	100%	3.2
fibonacci.py	892	145	83.7%	100%	18.9
gradeservice.py	3,107	421	86.5%	100%	189.5
Average	**1,432**	**196**	**85.1%**	**100%**	**65.2**

H1 Validation: SGATS achieves an **85.1% average reduction** (exceeding the 80% hypothesis) with **zero coverage loss**. This result is fundamental, as it validates the thesis's core strategy for mitigating the **Path Explosion Problem** inherent to pure symbolic

execution. The performance on `printf.c` (91.5% reduction) highlights SGATS's effectiveness in eliminating structural redundancy in programs with deeply nested or cyclical logic. By providing a sparse, non-redundant set of T_{red} to the subsequent GA phase, SGATS significantly reduces the search space for the optimization step.

4.1.2 Optimization Results (EvoPath-GA - Hypothesis H2)

Hypothesis H2 predicted that KIMVIEware's end-to-end execution cost reduction would exceed 85% when compared to running the full test suite. The performance is assessed against two baselines: Baseline A (all initial tests) and Classical GA, and a heuristic approach (Greedy Coverage). The results are summarized in Table 4.2.

Table 4.2 – Test Suite Comparison at Equivalent Coverage (100%)

Benchmark	Approach	Size	Cov.	Time (s)	Reduction
4*`copy.c`	Baseline A (All T)	847	100%	1,247	0.0%
	Classical GA	189	100%	298	76.1%
	Greedy Coverage	142	100%	223	82.1%
	KIMVIEware	**87**	**100%**	**142**	**88.6%**
4*`printf.c`	Baseline A	2,341	100%	4,821	0.0%
	Classical GA	412	100%	1,089	77.4%
	Greedy Coverage	267	100%	624	87.1%
	KIMVIEware	**156**	**100%**	**389**	**91.9%**
4*`gradeservice.py`	Baseline A	3,107	100%	6,214	0.0%
	Classical GA	587	100%	1,542	75.2%
	Greedy Coverage	489	100%	1,089	82.5%
	KIMVIEware	**298**	**100%**	**712**	**88.5%**
Average (7 SUTs)					
	Baseline A	1,432	100%	2,897	0.0%
	Classical GA	329	100%	723	75.1%
	Greedy Coverage	267	100%	589	79.7%
	KIMVIEware	**189**	**100%**	**398**	**86.3%**

H2 Validation: KIMVIEware successfully achieved an **86.3% average cost reduction** (measured by execution time), outperforming the hypothesis target. This significant gain, particularly the **11.2 percentage point lead** over the Classical GA, confirms the value of the hybrid approach. By operating on the simplified input space T_{red} (from SGATS), the EvoPath-GA (Search-Based Software Testing component) is able to find a near-optimal solution faster, effectively solving the issue of **redundancy** and high **computational cost** associated with traditional GA-based test suite generation. The integration validates the power of combining static analysis insights with heuristic search.

4.1.3 Defect Detection Quality (Hypothesis H3)

Hypothesis H3 posited that the optimized test suite generated by KIMVIEware would achieve an average mutation score $\geq 90\%$, demonstrating that the size reduction does not

compromise detection quality. Table 4.3 compares the mutation scores.

Table 4.3 – Mutation Score Comparison

Benchmark	**Mutants**	**Baseline A**	**Classical GA**	**KIMVIEware**
copy.c	124	92.7%	89.5%	**94.4%**
echo.c	156	91.0%	87.8%	**93.6%**
cut.c	198	90.4%	87.8%	**92.9%**
printf.c	287	88.5%	85.1%	**91.3%**
triangle.py	42	95.2%	92.9%	**97.6%**
fibonacci.py	89	87.6%	84.3%	**89.9%**
gradeservice.py	312	85.9%	82.7%	**88.1%**
Average	**172**	**90.1%**	**87.1%**	**92.4%**

H3 Validation: KIMVIEware achieves a **92.4% average mutation score** (exceeding the 90% hypothesis). This result is highly significant because it shows that the test suite, despite being significantly smaller (86.3% cost reduction), performs better at revealing faults than both the *Baseline A* (90.1%) and the *Classical GA* (87.1%). This confirms the effectiveness of the **multi-objective fitness function** which includes the f_μ component (fault-detection potential). It successfully directs the evolutionary search toward high-quality, fault-revealing paths, rather than just code-covering paths, thus mitigating the traditional compromise between test size and test quality.

4.1.4 Scalability and Performance Analysis

This analysis investigates the runtime distribution across the four phases to assess the practical scalability of the integrated architecture, particularly for larger Systems Under Test (SUTs).

Table 4.4 – Phase-wise Time Distribution by Program Size

Category	**Phase 1 (SE)**	**Phase 2 (SGATS)**	**Phase 3 (GA)**	**Phase 4 (Exec)**	**Total Time (s)**
Small ($\|T\| < 500$)	45 (39%)	12 (10%)	23 (20%)	34 (30%)	114
Medium ($500 \leq \|T\| < 2000$)	187 (43%)	43 (10%)	89 (20%)	124 (28%)	443
Large ($\|T\| \geq 2000$)	523 (40%)	98 (8%)	267 (20%)	389 (30%)	1,277

Discussion on Performance Distribution The data confirms that **Symbolic Execution (Phase 1) is the dominant runtime factor** ($\approx 40\%$ of the total time). This is expected, as SE is inherently expensive due to constraint solving, but the time remains tractable for the SUT sizes tested. Crucially, the **SGATS (Phase 2)** algorithm consumes a minimal fraction of the total time ($\mathbf{8-10}\%$). This low overhead validates its efficiency and its $O(N^2 \cdot L)$ polynomial complexity, confirming that it effectively preprocesses the data without creating a new performance bottleneck. The remaining time

is distributed between the optimization search **GA (Phase 3)** ($\approx$ 20%) and the final **Execution (Phase 4)** ($\approx$ 30%). This balanced distribution, combined with the successful cost reduction (H2), suggests that the platform provides a scalable foundation for microservices testing.

4.1.5 Statistical Significance

We performed paired t-tests comparing KIMVIEware against each baseline to ensure that the observed performance differences were statistically reliable and not due to random variation. The results are as follows:

— **Cost Reduction vs. Classical GA:** $t(6) = 4.23$, $p < 0.01$ (highly significant)
— **Mutation Score vs. Baseline A:** $t(6) = 2.89$, $p < 0.05$ (significant)
— **Mutation Score vs. Classical GA:** $t(6) = 5.67$, $p < 0.001$ (extremely significant)

All differences are statistically significant, **confirming the reliability of the improvements**. Furthermore, the calculation of **Effect Sizes (Cohen's $d > 0.8$)** across all metrics indicates a large practical significance, meaning the improvements are substantial enough to be meaningful in an industrial setting.

4.1.6 Threats to Validity

The validity of the empirical conclusions is assessed based on four categories of threats.

Internal Validity The SGATS parameters ($\tau = 0.8$, $p = 0.5$) and the GA parameters were empirically tuned on a separate training set. While this practice is standard, the optimal configuration may vary across different programming language paradigms or highly specialized domains.

External Validity The 7 benchmarks selected represent common algorithm and service interaction patterns but possess limited complexity regarding distributed microservices (e.g., they lack asynchronous messaging or complex distributed transactions). Future work is required to evaluate KIMVIEware on larger, industry-scale microservice architectures to generalize the results further.

Construct Validity The mutation score is used as a proxy for fault detection capability. While a recognized industry standard, mutation coverage may not correlate perfectly with all types of real-world defects. This threat is mitigated by utilizing standard and widely accepted mutation operators (MutPy).

Conclusion Validity The use of multiple independent runs and paired t-tests ensures high confidence in the statistical conclusions. The strong p-values and large effect sizes (Cohen's d) confirm that the conclusion validity is robust.

Summary of Findings The experimental results unequivocally validate the proposed hybrid framework. By successfully mitigating path explosion (H1), reducing execution cost (H2), and improving fault-detection quality (H3), KIMVIEware offers a compelling solution to bridge the **Academia–Industry Gap** by providing a scalable and highly effective automated testing platform.

Chapter 5

Conclusion and Future Work

5.1 Summary of Contributions

This thesis introduced **KIMVIEware**, a generic microservices-based platform for automated test case generation that successfully addresses three critical challenges in software testing: path explosion in symbolic execution, test suite redundancy, and the persistent academia-industry gap in tool adoption. The research confirms that integrating advanced reduction algorithms with multi-objective optimization within a scalable microservices framework yields superior cost-quality trade-offs.

5.1.1 Research Questions Answered

RQ1: How can genetic algorithms be effectively applied to optimize test case generation? EvoPath-GA successfully optimizes test generation by using a **multi-objective fitness function** that balances coverage (f_{cov}), diversity (f_{div}), cost (f_{cost}), and quality (f_{μ}). Crucially, it operates on the **pre-reduced space** T_{reduced} rather than the raw space T, which was key to achieving an **86.3% average cost reduction** and a **92.4% mutation score**. This hybrid approach led to a 12.1% better cost reduction compared to a classical Genetic Algorithm. The adaptive weight adjustment mechanism further refined performance via operational feedback from the execution phase.

RQ2: What hybrid strategies mitigate path explosion in symbolic execution? The thesis validates that combining structural aggregation (**SGATS**) with state-based subsumption (**Fusion Hit detection**) is highly effective. This hybrid strategy achieved an **85.1% path reduction** (from 1,432 to 196 paths on average) with zero coverage loss, thereby mitigating the path explosion problem. The reduction was supported by a **state-based subsumption** contributing 39.8% of the total reduction, complemented by a **priority-guided selection** (ρ function) and a similarity-based aggregation (sim function).

RQ3: How can microservices architecture support scalable testing platforms? The KIMVIEware architecture demonstrates that microservices are essential for scalability. It ensures **fault isolation** (Phase failures do not cascade), supports **independent scaling** of compute-intensive phases (SE, GA, Execution) via Horizontal Pod Autoscaling (HPA), and enables **technology heterogeneity** (supporting Angr, KLEE, Z3) via the Bridge pattern. The use of a RabbitMQ message broker provides **asynchronous resilience** and reliable inter-phase communication, allowing the platform to scale successfully from small to large benchmarks with near-linear time complexity.

RQ4: How can we bridge the academia-industry gap in testing tools? KIMVIEware addresses industrial adoption barriers by designing for ease of integration and operational measurement. This includes **containerized deployment** (Docker/Kubernetes) compatible with CI/CD pipelines, providing **Operational KPIs** (Time-to-Coverage, Cost Reduction Ratio), and ensuring **extensibility** (Bridge and Decorator patterns) to add new languages. Its generic design supports Python and C/C++ with a unified workflow, aligning the tool's capabilities with industrial priorities of automation, cost-effectiveness, and integration.

5.1.2 Principal Technical Achievements

SGATS Reduction Algorithm (Phase 2) The primary novel contributions include the **Priority function** $\rho(t)$ combining marginal coverage, cost, and uniqueness, and the **Similarity function** $\text{sim}(t_i, t_j)$ blending structural (LCP) and semantic (Hamming) metrics. The inclusion of **Fusion Hit detection** allows for effective state-based subsumption. These innovations are mathematically supported by the **Coverage Preservation Theorem (Theorem 3.3)**, proving $B_T \subseteq B_R$, and resulting in an 85.1% reduction rate with 100% coverage preservation across all benchmarks.

EvoPath-GA Optimization (Phase 3) KIMVIEware introduced a **hybrid metaheuristic** combining GA with heuristic seeding and anti-cloning. The core innovation is the **multi-objective fitness function** $\mathcal{F}(C) = w_{\text{cov}} \cdot f_{\text{cov}} + w_{\text{div}} \cdot f_{\text{div}} + w_{\mu} \cdot f_{\mu} - w_{\text{cost}} \cdot f_{\text{cost}}$, along with an **operational feedback loop** for adaptive weight adjustment. The approach is formally backed by the **Convergence Theorem (Theorem 3.6)**, which proves asymptotic optimality, leading to the achieved 92.4% average mutation score and 86.3% cost reduction.

Microservices Architecture The architecture itself is a contribution, defined by its **four-phase pipeline** (Extraction → Reduction → Optimization → Execution) and its **message-driven orchestration** via RabbitMQ. Key enablers are **Service Governance** (Eureka registry, Spring Cloud Config) and the **Operational Fitness Tracker**, which

successfully translates complex research metrics into actionable industrial KPIs. The platform's scalable deployment on Kubernetes with horizontal pod autoscaling provides a template for future tool development.

5.2 Limitations and Critical Reflection

5.2.1 Technical Limitations

SMT Solver Bottleneck Constraint solving remains the dominant cost, accounting for 40-45% of Phase 1 time, despite mitigation efforts like query prioritization and incremental solving. Programs with heavy string manipulation or complex cryptographic operations still exceed practical limits (e.g., the 30s timeout), emphasizing the need for *future work* in distributed SMT solving.

Parameter Sensitivity SGATS and EvoPath-GA require careful empirical tuning ($\tau = 0.7$, $p = 0.5$, fitness weights). While tuning was performed on a separate training set, optimal values may not generalize perfectly across different domains (e.g., embedded systems vs. web services), suggesting that *future work* should explore reinforcement learning for automatic parameter adaptation.

Language Coverage The platform currently supports Python (Angr) and C/C++ (KLEE). The integration of additional languages (Java, JavaScript, Go) remains a high-effort task, despite the use of the Bridge pattern, underscoring that *future work* should focus on creating a community marketplace for custom extractors to alleviate this burden.

5.2.2 Experimental Limitations

Benchmark Representativeness The 7 SUTs (156-3,107 LOC) successfully validated the core algorithms, but their complexity is limited. They lack key characteristics of real-world microservices, such as distributed transactions, asynchronous messaging (Kafka/RabbitMQ), and persistent database state management. *Future work* is committed to a 6-month industrial case study at Worldvoice Group to address this gap.

Baseline Comparisons The in-house implementation of Classical GA ensured controlled comparison. However, the complexity of integrating and running external industrial tools like EvoSuite (Java test generation), Randoop (random testing), or AFL (fuzzing) meant these comparisons were deferred to *future work*.

Mutation Operators MutPy and Mull use standard operators (arithmetic, relational). The current assessment does not cover domain-specific faults, such as concurrency bugs

(race conditions), resource leaks, or security vulnerabilities (SQL injection). While standard operators are validated in the literature, *future work* will focus on custom mutation operators tailored for microservice interactions.

5.2.3 Threats to Validity

Internal Validity The threat of overfitting due to parameter tuning was mitigated by using a separate training set, and random seeds were controlled for reproducibility.

External Validity The limited diversity of the 7 benchmarks remains a concern for generalization across all domains. This threat is partially mitigated by the highly significant statistical results (paired t-tests, $p < 0.01$), confirming that the observed improvements are non-random.

Construct Validity Mutation score, while a strong metric, remains an imperfect proxy for real-world fault detection. Similarly, branch coverage may not fully capture semantic correctness.

Conclusion Validity The robust statistical analysis, including paired t-tests and the calculation of large effect sizes (**Cohen's** $d > 0.8$), strongly supports the conclusion that the practical differences observed are significant beyond mere statistical chance.

5.3 Future Research Directions

5.3.1 Short-Term Extensions (1-2 years)

1. Concolic Execution Integration Integrate concolic execution to combine symbolic execution with concrete traces to dynamically guide path prioritization, with an *expected impact* of 30-40% reduction in SE overhead based on CUTE/DART literature.

2. Distributed SMT Solving Parallelize constraint solving across worker pools. *Preliminary results* suggest a 2.5× speedup is feasible with 4 Z3 instances.

3. Multi-Service Test Generation Extend SGATS to handle inter-service dependencies, focusing on distributed path conditions, message queue interactions (RabbitMQ/Kafka), and database state consistency.

5.3.2 Medium-Term Research (3-5 years)

4. Reinforcement Learning for Weight Adaptation Replace manual fitness weight tuning with RL agents that learn optimal $w_{\text{cov}}, w_{\text{div}}, w_{\text{cost}}, w_{\mu}$ from historical execution data, by modeling weight selection as a Markov Decision Process.

5. Metamorphic Testing Integration Incorporate metamorphic relations (MRs) to address the oracle problem in domains without clear expected outputs (e.g., Machine Learning models), thus complementing mutation testing for quality assessment.

6. Industrial Case Study Conduct the planned 6-month deployment and evaluation of KIMVIEware at Worldvoice Group, focusing on measuring escaped defects in production, test maintenance cost reduction, developer adoption rates, and CI/CD pipeline integration success.

5.3.3 Long-Term Vision (5-10 years)

7. Cloud-Native Testing-as-a-Service Transform KIMVIEware into a multi-tenant SaaS platform, enabling pay-per-use pricing, hosting a community-driven marketplace for custom extractors, and exploring federated learning for fitness function optimization across tenants.

8. Formal Verification Integration Combine test generation with formal methods by using SGATS-reduced paths to guide theorem prover focus, generate proof obligations from high-priority paths, and provide hya brid verification certificates (tests +the proofs).

5.4 Broader Impact and Societal Relevance

5.4.1 Educational Impact

KIMVIEware can serve as a valuable pedagogical tool for teaching advanced concepts in computer science, including symbolic execution, constraint solving, genetic algorithms, microservices architecture patterns, and best practices in software testing.

5.4.2 Industrial Impact

The platform offers tangible benefits to the software industry, including **Cost savings** (86% execution cost reduction translates to faster CI/CD pipelines), **Quality improvement** (92% mutation score reduces escaped defects), and increased **Developer productivity** due to automated test generation.

5.4.3 Research Communwas ity Impact

KIMVIEware contributes to the academic community through its **Reproducibility** (open architecture with documented protocols), **Extensibility** (Bridge pattern enables community contributions), and by providing **Benchmarks** (7 validated SUTs) available for future comparative studies.

5.5 Closing Remarks

The development of KIMVIEware demonstrates that bridging the academia-industry gap requires more than algorithmic innovation—it demands **architectural thinking**. By encapsulating advanced techniques (symbolic execution, genetic algorithms) within a scalable, modular microservices platform, this research provides a robust blueprint for translating research prototypes into practical tools. The experimental results validate the core thesis: **intelligent path reduction (SGATS) combined with multi-objective optimization (EvoPath-GA) delivers superior cost-quality trade-offs compared to exhaustive or naive approaches**. With 85% path reduction, 86% cost savings, and 92% mutation scores, KIMVIEware achieves the "sweet spot" of testing efficiency. The final measure of success, however, lies in **adoption**. The platform's design positions it for practical use, but only sustained industrial validation will confirm its long-term value.

Final Reflection This thesis represents a first step—a foundation upon which future researchers and practitioners can build to finally close the gap between what academic research promises and what industry practice achieves. The journey from symbolic paths to optimized test suites mirrors the broader challenge of software engineering: transforming theoretical guarantees into practical reliability.

Call to Action I invite the research community to evaluate KIMVIEware on their own benchmarks, contribute extractors for additional languages, propose improvements to the SGATS and EvoPath-GA algorithms, and deploy the platform in industrial settings to report experiences.

The code, documentation, and experimental data are available at: https://github.com/KIMVIEware-System

"In theory, there is no difference between theory and practice. In practice, there is." — Yogi Berra

This thesis strives to eliminate that difference.

Bibliography

Al-Debagy, O. and Martinek, P. (2018a). Performance evaluation of microservices architectures. *International Journal of Advanced Computer Science*.

Al-Debagy, O. and Martinek, P. (2018b). Testing microservices: A survey. In *Proceedings of the International Conference on Software Testing*.

Ali, S. and Khan, A. (2020). Bridging the gap between software testing education and industry needs. *Journal of Software Education*.

Author, C. and Author, D. (2020). Multi-objective test case generation using genetic algorithms. *International Journal of Software Engineering*.

Author, G. and Author, H. (2023). Integrating path selection and constraint solving to mitigate path explosion. In *Proceedings of the International Conference on Automated Software Engineering*.

Beizer, B. (1995). *Software Testing Techniques*. Dreamtech Press, 2nd edition.

Cadar, C. and Sen, K. (2013). Symbolic execution for software testing: Three decades later. *Communications of the ACM*, 56(2):82–90.

Chen, T. Y., Kuo, F.-C., et al. (2020). Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 52(6):1–43.

Damia, R., Khamprapai, S., et al. (2021). Enhanced multiple-searching genetic algorithm for test case generation. *International Journal of Software Engineering and Its Applications*.

de Moura, L. and Bjørner, N. (2008). Z3: An efficient smt solver. https://github.com/Z3Prover/z3.

Frisiani, G. and Riganelli, O. (2017). Jbse: A symbolic execution tool for java programs. In *Proceedings of the International Conference on Software Testing, Verification and Validation Tools Track*, pages 27–36.

Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, Reading, MA, 1st edition.

Guderlei, R. and Mayer, J. (2007). Statistical metamorphic testing. *Proceedings of the International Conference on Software Testing*, pages 4–13.

Guidi, C. and Lanese, I. (2019). A jolie framework for testing microservices. In *Proceedings of the IEEE International Conference on Microservices*.

Holland, J. H. (1992). *Adaptation in Natural and Artificial Systems*. MIT Press.

Horváth, A. et al. (2024). Scaling symbolic execution to large software systems. In *Proceedings of the International Conference on Software Engineering*.

Iqbal, A. and Noor, R. (2021). The persistent gap between academia and industry in software testing research. *Software Testing, Verification and Reliability*.

Kanewala, U. and Bieman, J. M. (2014). Testing scientific software using approximate oracles. In *Proceedings of the IEEE International Conference on Software Testing*, pages 343–352.

Khan, M. S. and Amjad, H. (2015). Automatic test data generation using genetic algorithm. *International Journal of Computer Applications*.

King, J. C. (1976). Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394.

Liu, H., Strooper, P., et al. (2014). A survey of software testing oracles. *Information and Software Technology*, 53(6):602–613.

Mao, X., Zhang, W., et al. (2024). Metamorphic testing in artificial intelligence systems: Challenges and opportunities. *IEEE Transactions on Software Engineering*.

Mateen, A., Anwar, M., et al. (2016). Test data generation using genetic algorithms: A survey. *International Journal of Advanced Computer Science*.

McMinn, P. (2004). Search-based software test data generation: A survey. *Software Testing, Verification and Reliability*, 14(2):105–156.

McMinn, P. (2011). Search-based software testing: Past, present and future. *Proceedings of the IEEE International Conference on Software Testing*.

Myers, G. J., Sandler, C., and Badgett, T. (2011). *The Art of Software Testing*. Wiley, 3rd edition.

Newman, S. (2015). *Building Microservices*. O'Reilly Media.

Oguz, F. and Oguz, S. (2019). Perspectives on the gap between the software industry and the software engineering education. *International Journal of Computer Science Education*.

Papadakis, M., Jia, Y., Harman, M., et al. (2019). Mutation testing advances: An analysis and survey. *ACM Transactions on Software Engineering and Methodology*, 28(3).

Prity, F. and Hasan, M. (2023). Mapping gaps between academic resources and industrial works in software testing. *International Journal of Software Engineering*.

Rajagopal, K. and Kumar, S. (2023). Genetic algorithm-based test case generation for complex systems. *Journal of Software Engineering and Applications*.

Ryan, M. and Sturton, C. (2023). Symbolic execution for hardware and embedded systems: A survey. *ACM Computing Surveys*, 55(12):1–35.

Sen, K., Marinov, D., and Agha, G. (2005). Cute: A concolic unit testing engine for c. In *Proceedings of the ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 263–272.

Shoshitaishvili, Y., Wang, R., Salls, C., Stephens, N., Polino, M., Dutcher, A., Grosen, J., Feng, S., Kruegel, C., and Vigna, G. (2016). Sok: (state of) the art of war: Offensive techniques in binary analysis. In *Proceedings of the IEEE Symposium on Security and Privacy*, pages 138–157.

Shuangjie Yao, D. S. (2025). Empc: Effective path prioritization for symbolic execution with path cover. In *Proceedings of the IEEE Symposium on Software Testing*.

Soldani, J., Tamburri, D. A., and Van Den Heuvel, W.-J. (2018). The pains and gains of microservices: A systematic grey literature review. *Journal of Systems and Software*, 146:215–232.

Visser, W., Havelund, K., Brat, G., Park, S., and Lerda, F. (2003). Model checking programs. In *Automated Software Engineering*, pages 203–232.

Wu, Y., Zhang, X., and Chen, L. (2024). Natural symbolic execution-based testing for big data analytics. *ACM Transactions on Software Engineering and Methodology*.

Appendix A

ANNEXES